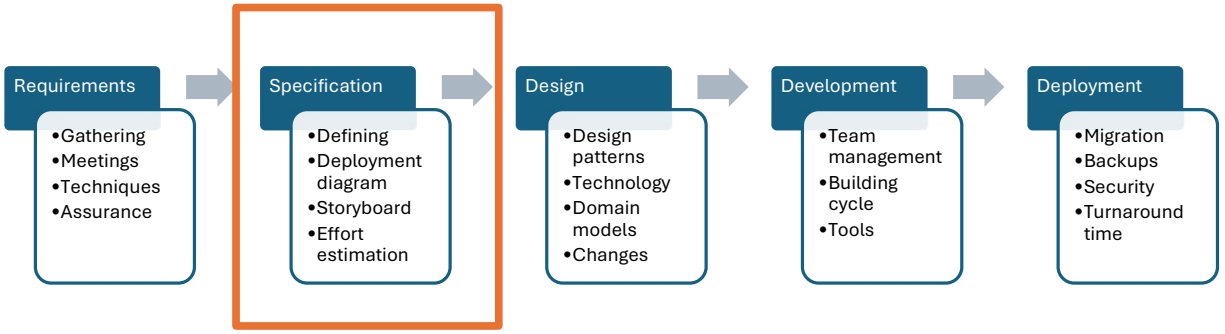
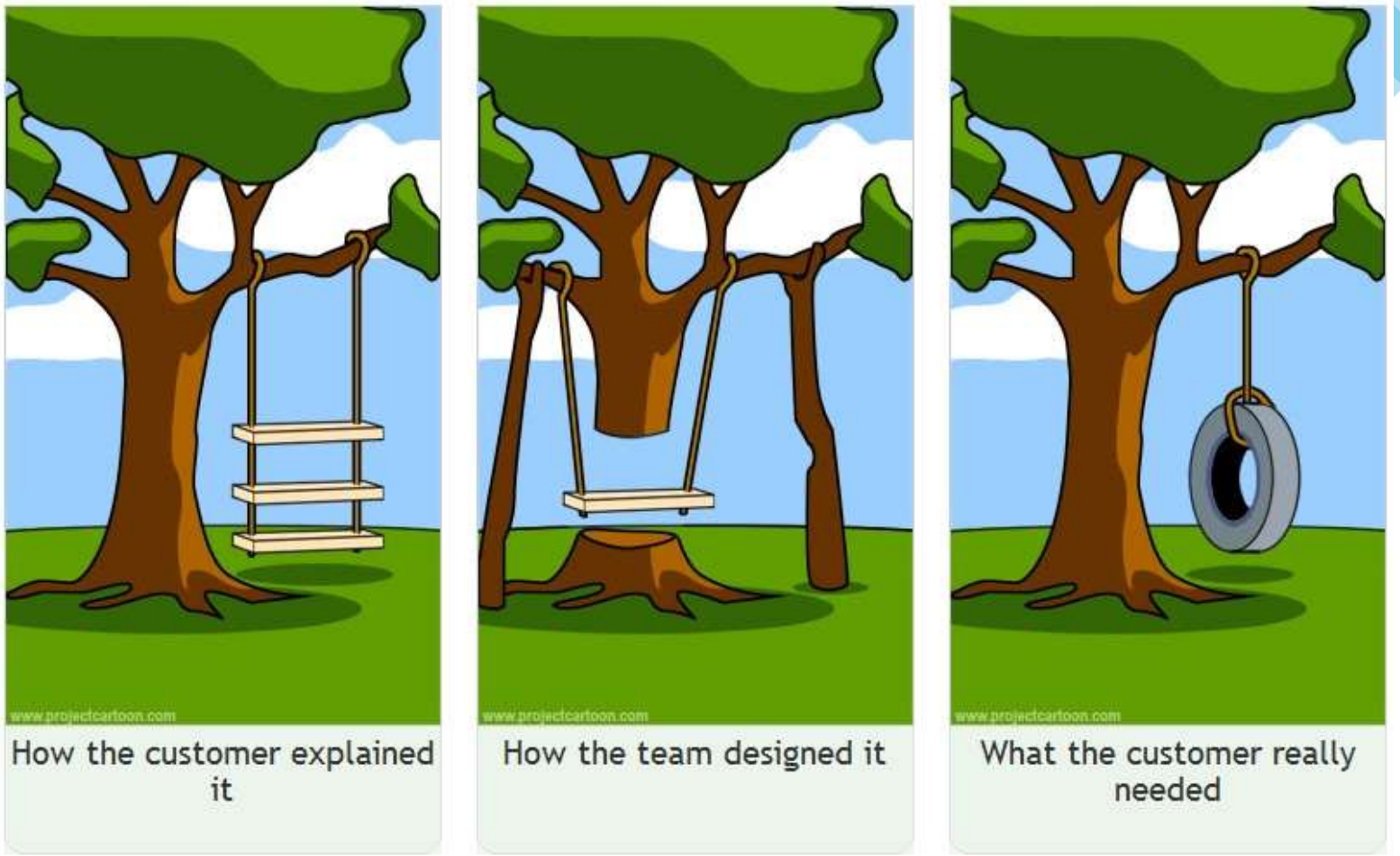


Specifications

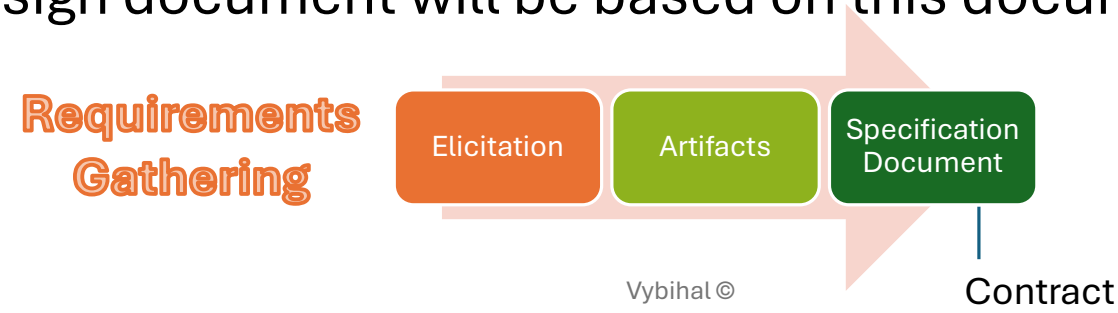
The Software Engineering Process: Phase 2


How the customer explained it
How the team designed it
What the customer really needed

What is the specification document?

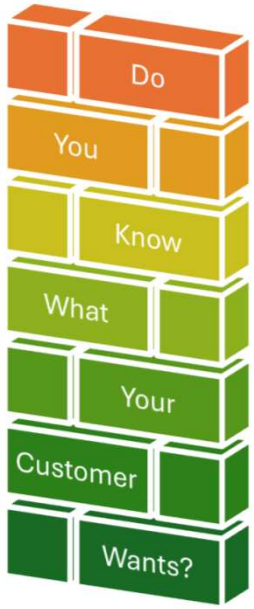
- Converting the artifacts from the elicitation phase into a set of formal complete diagrams.
- Developing a non-technical document that fully describes the project from the point of view of the client, using their language, but structured using engineering principles.
- The client will sign-off on the project based on this document.
  - This document will form a portion of the contract
- The design document will be based on this document.

Bridge document



Important diagrams

- The Storyboard
- Screen Layouts
- Database and File Schemas
- Flowcharts of critical (important) areas (not pseudo code)
- Simple Deployment Diagram



Purpose

The purpose of the specification document is to formalize and finalize the expectations of the user before we plan the data structures and algorithms of the actual app.

Title Page - Contents - Resources

Title Page	Table of Contents	Resources

Specifications

Project Name

Date

Author(s) & IDs
Company Name

Contents

1. Background . . 1
    1. Topic A . . 2
    2. Topic B . . 3
2. Overview . . 5
    1. Topic A . . 6
    2. Topic B . . 7
    3. Topic C . . 8
3. Definitions . . 10
4. Screen Layouts . 15
5. Schemas & Queries . 20
6. Critical Sections . 25
7. Effort Estimation . 30
8. Resources . . 35

Resources

Appendix A: References
Appendix B: Photocopies
Appendix C: Templates/Forms
Appendix D: Digital Source Locations
Appendix E: Index

Each resource section is a titled page with either a table (e.g., digital source locations), a list (e.g., references, index), or a series of attached pages (e.g., photocopies, templates, forms).

Additional appendices can be added as needed.

Strict formatting standards

- Numbered chapters
- Numbered subsections
- Numbered diagrams
- Standard report writing literary style
  - But we try to be concise in the length of our dialogues, and we favour the use of diagrams to augment our paragraphs.

Never create a diagram without at least one paragraph explaining each part of the diagram.

DIAGRAM
PARAGRAPHS

1 Background



1.1 Motivation

- Why is this being developed?
- What problem is it trying to make better (or solve)?
- Who primarily asked for it, and why?
- Other purpose-related information to keep in mind?

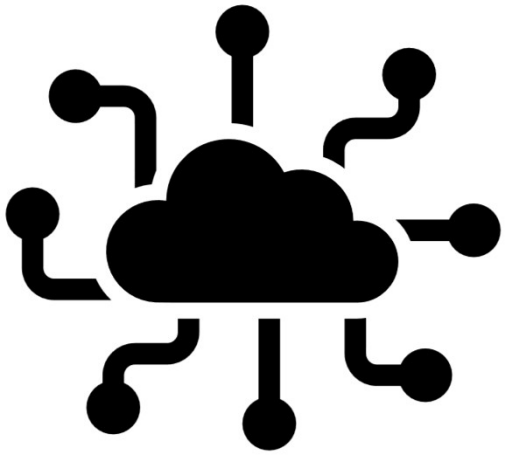
1.2 Stakeholders

Type	Name	Coordinates	Success Measure
Owner	M. Papadopoulos	Phone number, email, and/or office number.	If it does x, I call the application a success!
Manager	J. Chan		
End User Employee	Gregory (cashier)		
End User Customer	Lenora Wilmington		
Indirect User	Fedora Felegund		

Does not use, but impacted

Question:
What is the motivation/purpose of app?

- Difference between the motivation:
  - YouTube vs.
    - Keep customer's (users) engaged with the platform to consume advertisement and be entertained.
  - Personal Homepage, vs.
    - Provide information to the public about the owner of the home page.
  - Web-based email client.
    - Simplify email management and creation.



Question:
Who are the stakeholders?

- Identify stakeholders:
  - Grocery store cash register, vs.
    - Simplify the life of the **teller**
    - Process **customers** quickly
    - Record and process the sale for the **manager** and **accountant**
  - Grocery store self-checkout registers.
    - Easy for the **customer** to process their own items
    - Simplify the payment and rewards process for the **customer**
    - Record the sale for the **manager** and **accountant**
    - Provide easy issue management for the **staff**



2 Overview

2.1 Usage

- Table of how/when each stakeholder will use the application.
- Description of the way each stakeholder launches/accesses features of the program.

2.2 Project

- Scheduling
  - Expected development start and completion dates
- Architecture
  - Storyboard diagram
  - Deployment diagram

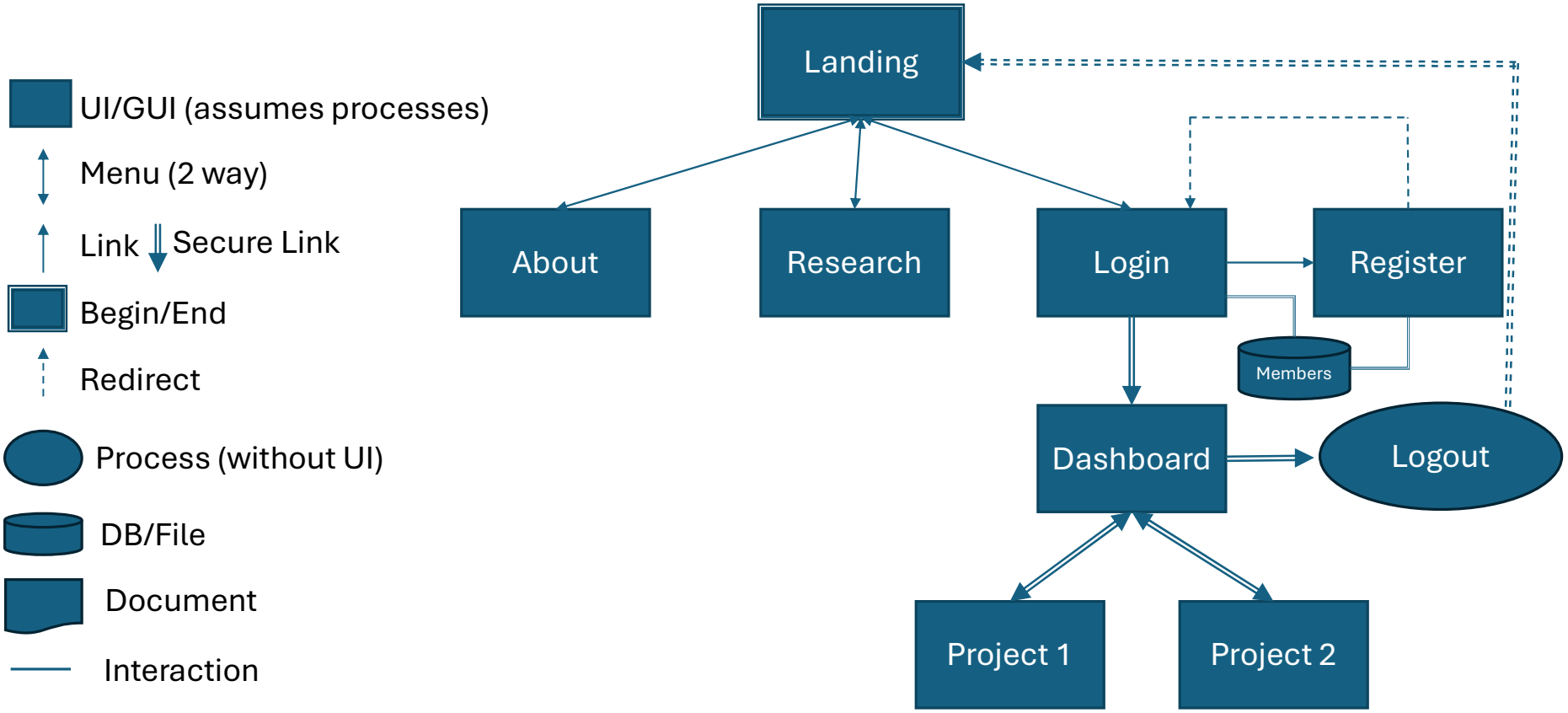
What is the usage and access table for…

- A self-server cash register that handles cash and credit. The user can scan products or use a keyboard to input product codes. The keyboard is also used to enter discount vouchers.
  - Identify stakeholders (direct/indirect/manager/owner/technicians)
  - How/when used
  - Describe specifically how to launch/access

The Storyboard

- A high-level architectural flowchart of the application in its entirety
- Presented from the user's point-of-view
  - What will the user interact with when they launch and use the program, highlighting the files/databases/network resources that are accessed/changed during that interaction.
- It is presented as a graph of all possible paths.
  - Each node: UI or top-level background process (not object/function).
  - Each arc: a labelled trigger moving the user to the next node.

Storyboard Example: website



Storyboard Example: telescope app

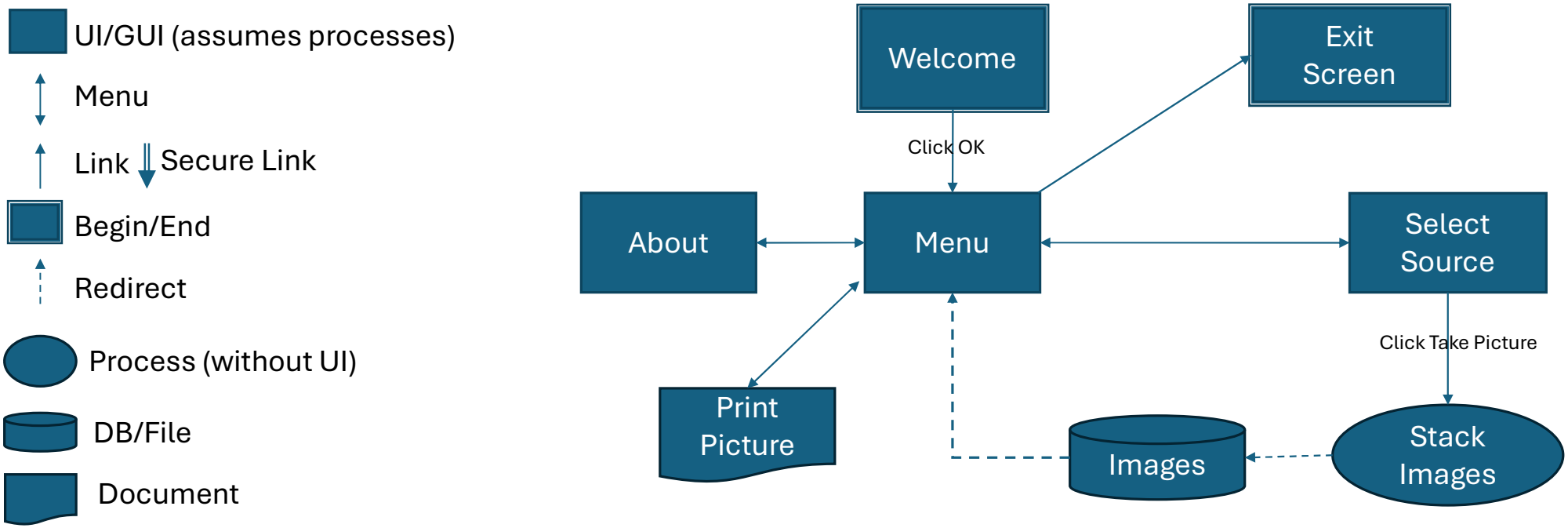


Diagram Presentation

SECTION TITLE

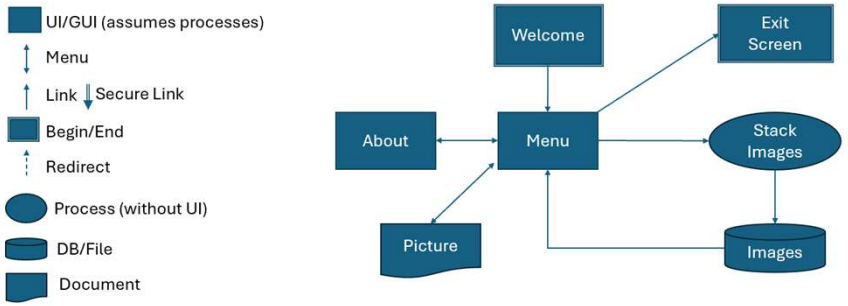


Figure x: description

Paragraph about Welcome

Paragraph about Menu

Paragraph about Stack Images

etc.

The goal is to capture a single, comprehensive view of the entire application on a single page from the client's perspective.

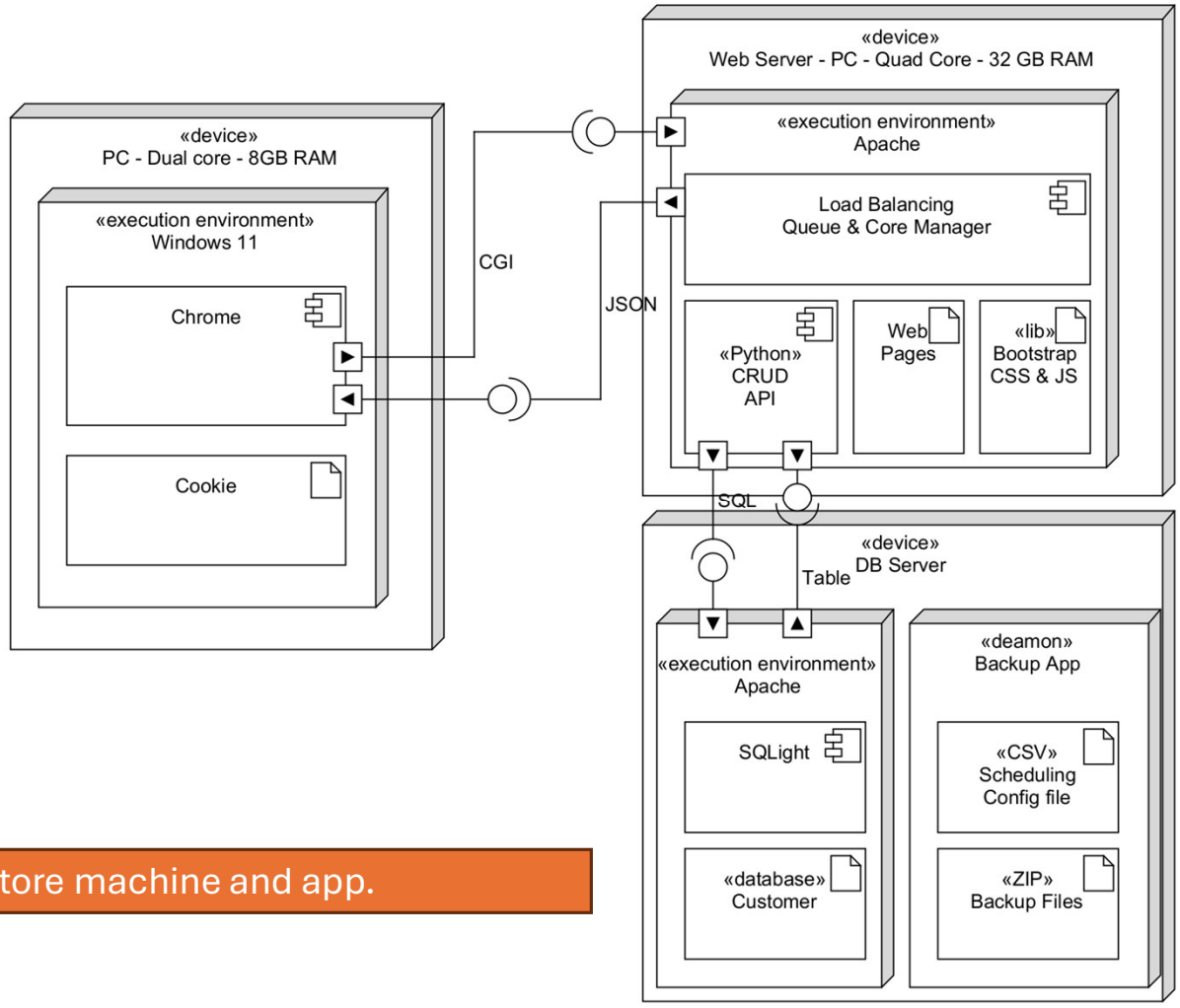
Diagrams always make sense to the author, but rarely do for the reader. It is essential to explain to the reader how they should interpret the diagram.

Deployment Diagram

Goal: Describing what the program will look like when installed in the client's infrastructure.

- Devices
- Execution environments
- Components
- Documents
- Communication channel

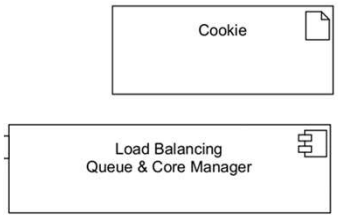
Describe things completely.



Q: List the parts of a self-checkout grocery store machine and app.

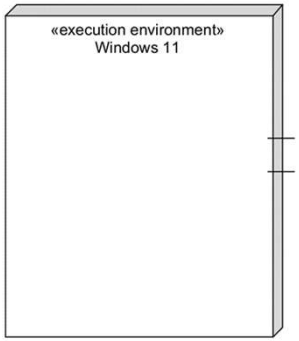
Deployment Diagram Elements

Notice how everything is labeled using **much technical detail**.

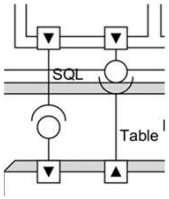


File

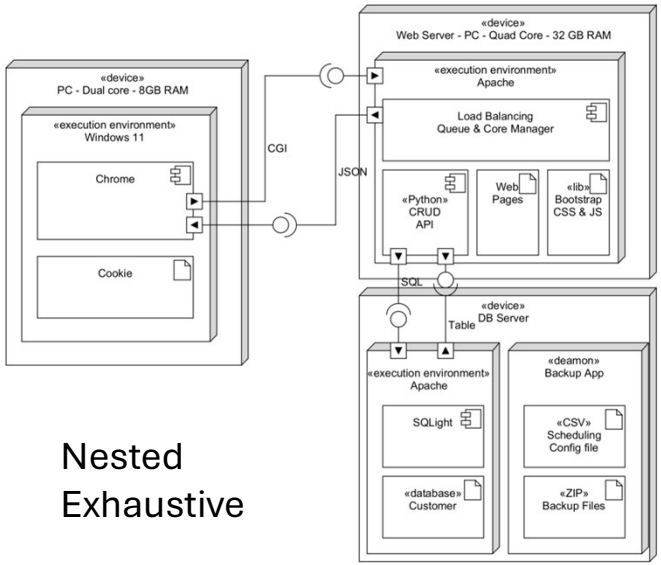
Program



Runtime environment.
Device, OS, Interpreter.



Labeled communication paths.
Ball & socket with direction arrow.
Used to express:
- Network communication
- Local API calls



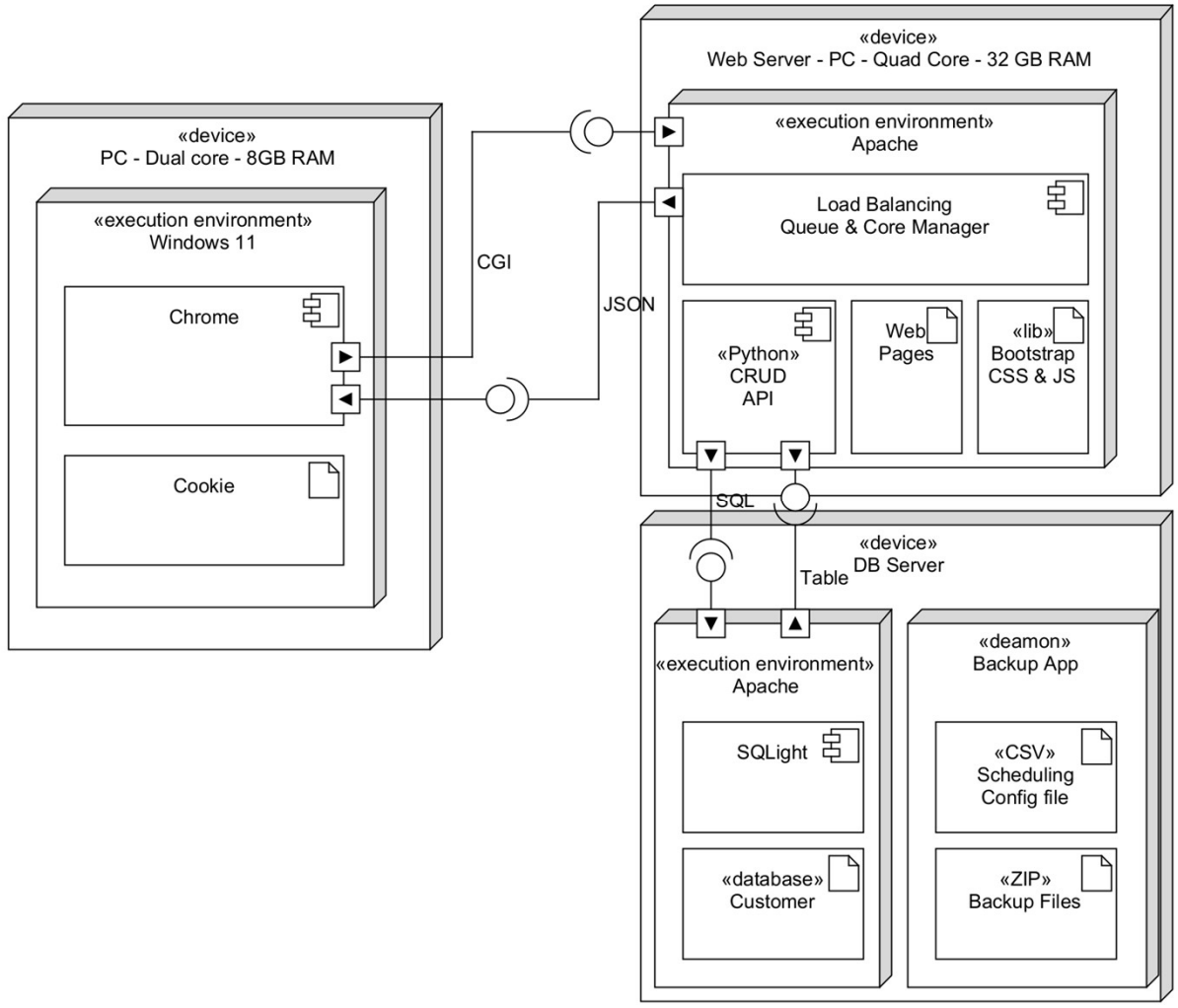
Nested
Exhaustive

Deployment Diagram

Question

What does this diagram show?

Is anything wrong?



Notice the servers do not have an operating system. This is a problem, unless any operating system will do. Note: any OS means Windows, Mac, Linux, Unix, Minix, Android, etc. (are you sure about that?)

Diagram Presentation

SECTION TITLE

Diagram Key

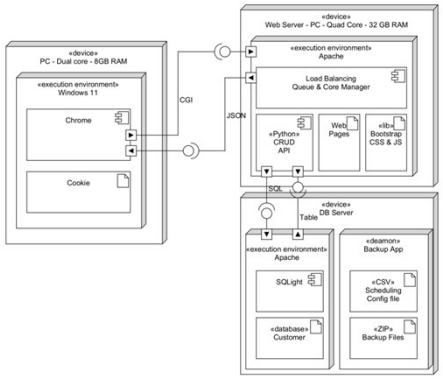


Figure x: description

Paragraph about component 1

Paragraph about component 2

Paragraph about Stack component 3

etc.

The goal is to capture a single, comprehensive view of the entire application on a single page from the client's perspective.

Diagrams always make sense to the author, but rarely do for the reader. It is essential to explain to the reader how they should interpret the diagram.

Useful Tools

- For design diagrams
  - UML ET
  - Figma



3 Definitions

- 3.1 Branding
  - Logos, Color palette
  - Look and Feel
- 3.2 Required Technologies
  - Devices, programming languages, libraries, run-time environment
  - Compute, Computers, Servers, Network
- 3.3 Project and System Constraints
  - Budget limitations, Equipment limitations, Tech. limitations, etc.
- 3.4 Client Expectations
  - Must have features, Expectations / Wants
  - Assumptions, What is considered success/failure
  - Project important won't and will do
- 3.5 Impact/Ethics/Performance Statements

Other than 3.1 and 3.5, these are mostly point lists or tables.

Cannot leave blank or skip.

We have seen the Impact, Ethics, and Performance statements previously.

NOTE
Client – person you are building the program for.
Customer – The people the client is servicing or selling to.

3.1 Branding



https://visme.co/blog/website-color-schemes/
Color pallet and sample page in branding.

- Describe look and feel
- Present the color pallet
- Present the logo
- Provide a sample image containing the above
- Justify why you chose this

Logo maker: https://www.design.com/s/logo-maker

Example

- What might we suggest as branding for Hema Quebec?
  - https://www.hema-quebec.qc.ca/index.en.html



Branding

Tools

- https://www.design.com/s/logo-maker
- https://colors.muz.li/

Look and Feel

- How should the screens transition? Slide from side?
- The user is experienced so present it as a dashboard app.
- The user needs a friendly step by step dialog prompting app.

3.2 Required Technologies (by example)

- Factory offices
  - 4 Windows computers
  - 1 MacBook Pro with printer for reports
  - 1 Windows server running Ethernet
  - Building one app, must be compatible.

ABC Inc.'s office is based on Windows technology. Their workstations and server run Windows; however, the owner uses a MacBook Pro when away. The app needs to support this environment.

- Factory plant
  - 1 Windows PC employees use to punch-in and out their hours
  - 1 Windows PC for network supervisor
  - 1 Windows PC for the CAD drafts person
  - All connected to the Windows server in the Factory Offices
  - App needs to be compatible
- Software
  - Intranet using PHP and SQL.

ABC Inc.'s office and factory are not connected to the Internet. They have a LAN that runs an Intranet and a single common application called Gem that everyone uses for the business.

3.3 Project and System Constraints (by example)

- Budget limitations: < $50,000.00.
- Equipment limitations: cannot add an additional server.
- Tech. limitations: project must run on the browser only.

Expectations

3.4 Client Expectation

- Must have features, Expectations / Wants
- Assumptions, What is considered success/failure
- Project important won’t and will do

3.5 Statements

- Formal writeup of what we spoke about during the gathering of requirements phase: impact, ethics, and performance.

4 Screen Layouts

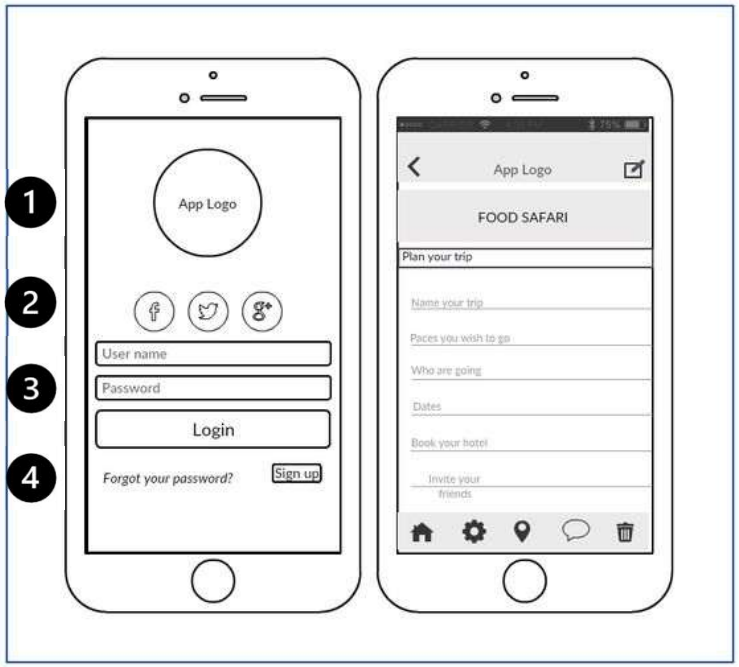
- 4.1 Screen Mockups
  - Indexed by Storyboard naming
  - All Storyboard UI / GUI screens need to be defined
- 4.2 Specialty screens: (not depicted in Storyboard)
  - Welcome / Splash screen
  - Terminate / Outro screen
  - Error / Warning screen(s)
- Notes:
  - Not every screen
  - Drawn following branding

This is important so that the client understands what they will be receiving. It also gives them a chance to tell you what they like and don't like about your UI.

Elements of a screen layout

- A realistic diagram of the screen
- Presented in the agreed branding
- Numbered bubbles permit you to identify and describe parts of the mockup.
- A table with the descriptions.

Screen Mockup

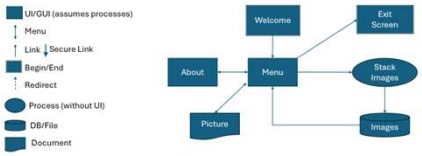


Description Table

1. Pressing on the circle permits the user to select the picture. If it has already been selected, it displays automatically.
2. Pressing the social media icons launches the social media app already existing on the device.
3. The user must press login after entering their username and password to get access to the next screen (as shown on the right).
4. If the user forgot their password and has not yet created an account, two clickable text buttons are at the bottom of the screen.

Formatting this section?

SECTION TABLE OF CONTENTS



List boxes as “mockup x” and why.

MOCKUP 1 - LOGIN



Description Table			

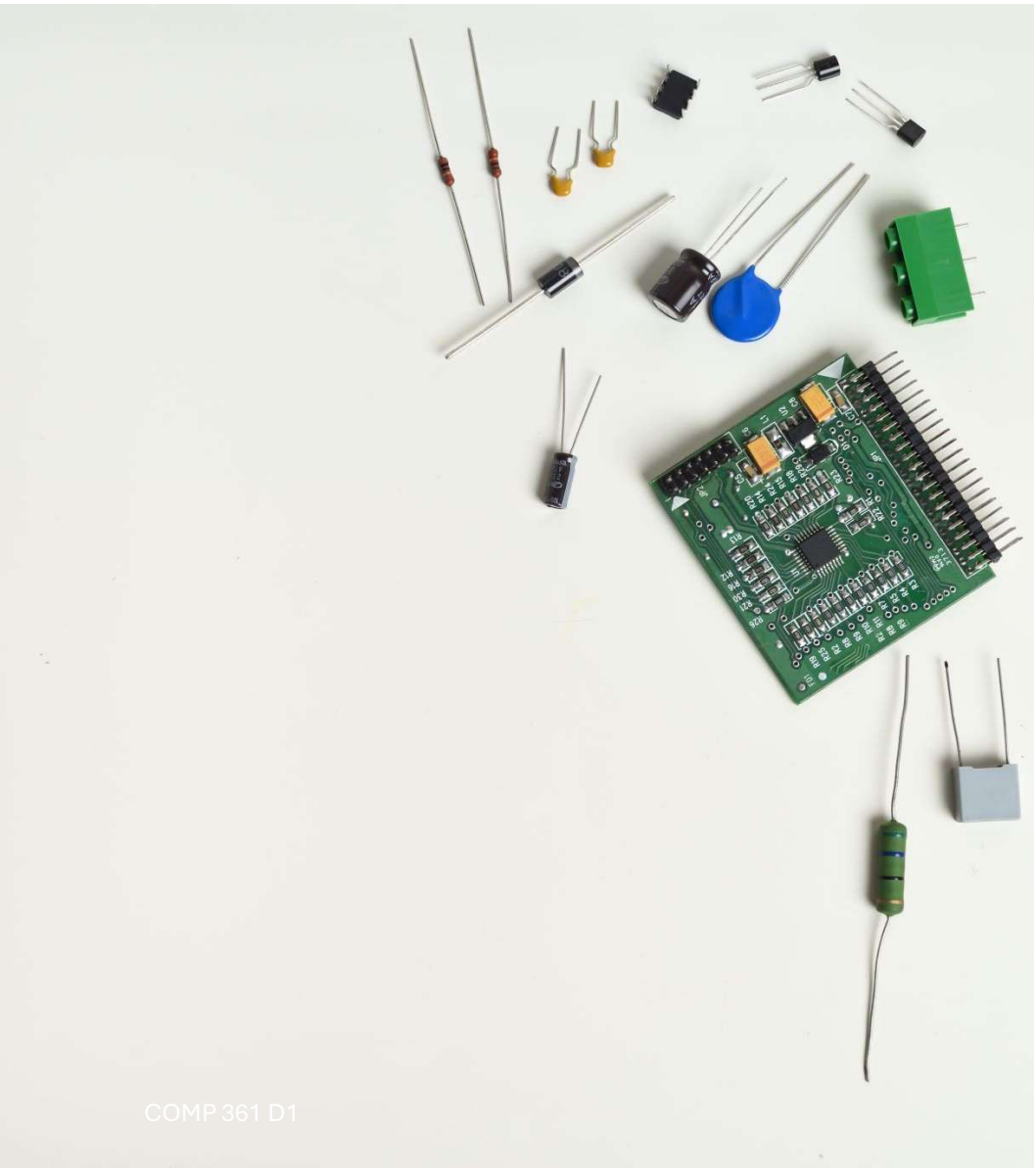
MOCKUP 2 – MAIN PAGE



Description Table			

Present a small image of the storyboard as a sort of “table of contents” for this section. Identify the boxes that will be expanded upon below.

Do not combine this into a single image (for example, change the storyboard into mockups).



Useful Tools

- What I like to do is to build the GUI in the language of choice for the project (without any “meat”). It becomes a good starting point for the developers & the user can play with it.

- For screen layout design:
  - Balsamiq, or
  - Figma

5 Schemas & Queries

Important to capture
- Relationships
- Rules
- Structure

- 5.1 Defining file structures:
  - 5.1.1 Database diagram
  - 5.1.2 Database table structures
  - 5.1.3 Formatted text files (like CSV, JSON, XML, INI, custom)
  - 5.1.4 Defined Operations (defining operations on those files)
    - Queries, using SQL expressions for the formality and precision of description.
    - Returned value format
- 5.2 Remote API definitions
  - API signatures
  - returned value format

Purpose

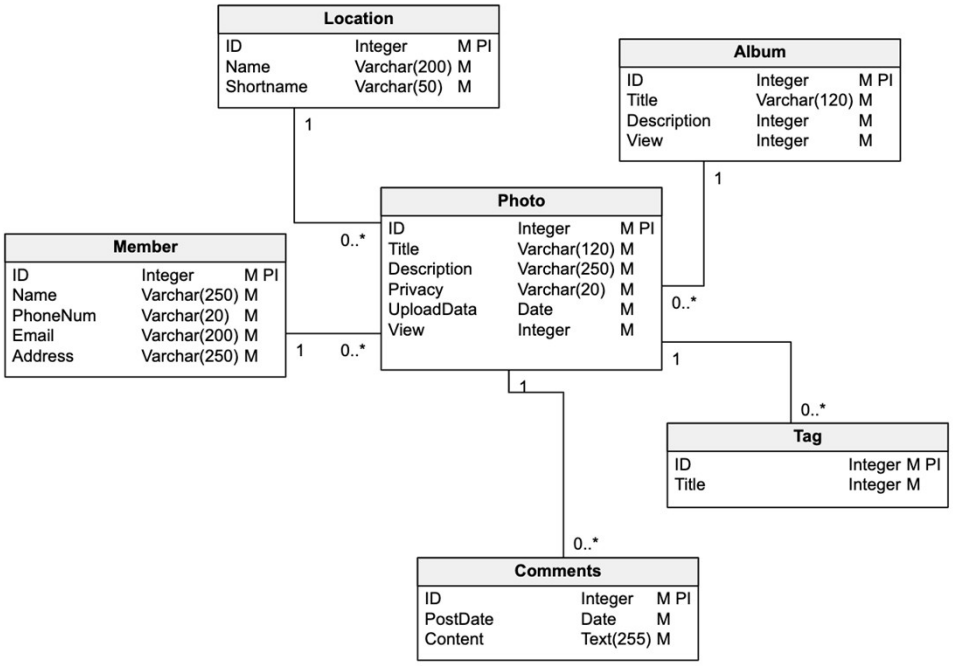
- Why do we need this data?
- How should we store this data?
- How should we use this data?
- Are there any rules or restrictions associated with this data?



The goal is to make it all fit on a single page.

Database Diagram

- A method by which you can display all the files/databases in one image.



Simple diagram would not contain the field names. Best use-case when database is very large. Need to augment with tables.

Table/file name
Fields & type info.
Associations & counts

The example shown here is best for smaller sized databases. It can be all-in-one.

M = member
PI = Primary Key
K = Key
F = Foreign member

Database table structure

Membership Table

Filed Name	Type	Length	Key, Primary, Sort	Constraints	Legal Values
First_Name	String	30 characters		Alpha, space, hyphen	
Last_name	String	50 characters	Primary key, Alphabetical order	Alpha, space, hyphen	
Age	Int	3 digits		Valid range 0 to 120	

For complex and large databases, separating the database diagram from the table permits you to define additional elements like Constraints and legal values.

Question

- What might a database diagram look like for managing the students in a course?
  - What information do we need?
  - Can the information be divided into fields and tables?
  - How are the tables associated with one another?
  - What things do we need to edit?
  - What things do we need to display / report on?
  - Who should have access to what information?
  - Are there any validation rules or legal value ranges?

Formatted text file structures

- Present each file in the standard way one represent the file type.

- Here are two examples:
  - CSV Header:
    - firstName, lastName, age
  - XML Header:
    - <Membership>
      <firstName type="string" constraints="alpha space hyphen"> </firstName>
      <lastName type="string" constraints="alpha space hyphen"> </lastName>
      <age type="integer" constraints="range:0 to 120"> </age>
      </Membership>

Question

- Assume we have a machine that outputs date, time, and temperature measurements to stdout as a space-separated string with each measurement on its line.
- In which file format should we store this information?
  - Is there an optimal file format?
  - Does the file format depend on our purpose? Why?
  - Are there formats to avoid?
- How should the file's contents be formatted?

Operations on files

Queries:

SQL	Motivation
Select * from Membership	List all members
Select Last_name, First_name from Membership where Last_name=[input]	Search by last name
Select * from Membership where age<18	Search for underaged members

All queries are expressed as SQL statements, even though you may not implement the code as SQL. Presenting it in SQL is a formal but readable representation most clients can easily understand. It is also useful because most developers can also understand it.

Remote API Definition

The Design Document will have a better section for this, but from the user's point of view it is just another type of query.

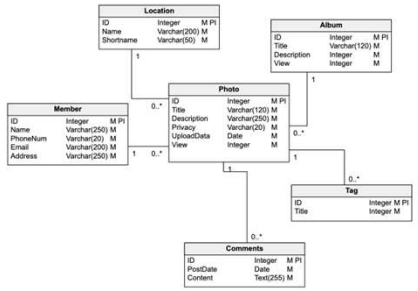
- API call signature
  - GET membership/list
  - GET membership/search vybihal
- API description and usage rules
  - The user must input a valid last name. If it is not formatted correctly and error message will result. A JSON string is returned to the caller.
- API return structure (normally JSON or CGI or database table)
  - { "listing" :{ {"fname":"bob", "lname":"smith", "age":15},
    {"fname":"jane", "lname":"doe", "age":20}
    }
    }

Question

- What would be the API description for login and register?

Formatting this section?

SECTION TABLE OF CONTENTS



Membership Table

Filed Name	Type	Length	Key, Primary, Sort	Constraints
First_Name	String	30 characters		Alpha, space, hyphen
Last_name	String	50 characters	Primary key, Inc.	Alpha, space, hyphen
Age	Int	3 digits		Valid range 0 to 120

Queries:

SQL	Motivation
Select * from Membership	List all members
Select Last_name, First_name from Membership where Last_name=[input]	Search by last name
Select * from Membership where age<18	Search for underaged members

Membership Table

Filed Name	Type	Length	Key, Primary, Sort	Constraints
First_Name	String	30 characters		Alpha, space, hyphen
Last_name	String	50 characters	Primary key, Inc.	Alpha, space, hyphen
Age	Int	3 digits		Valid range 0 to 120

Queries:

SQL	Motivation
Select * from Membership	List all members
Select Last_name, First_name from Membership where Last_name=[input]	Search by last name
Select * from Membership where age<18	Search for underaged members

API BY AREA

MEMBERSHIP

Signature:

GET membership/list

GET membership/find [last_name]

Description:

Talk about each signature syntax

Requirements for input

Expected response time, if needed.

Return format:

Complete syntax of return string

Provide at least one example

TRANSACTIONS

Signature

GET

Description

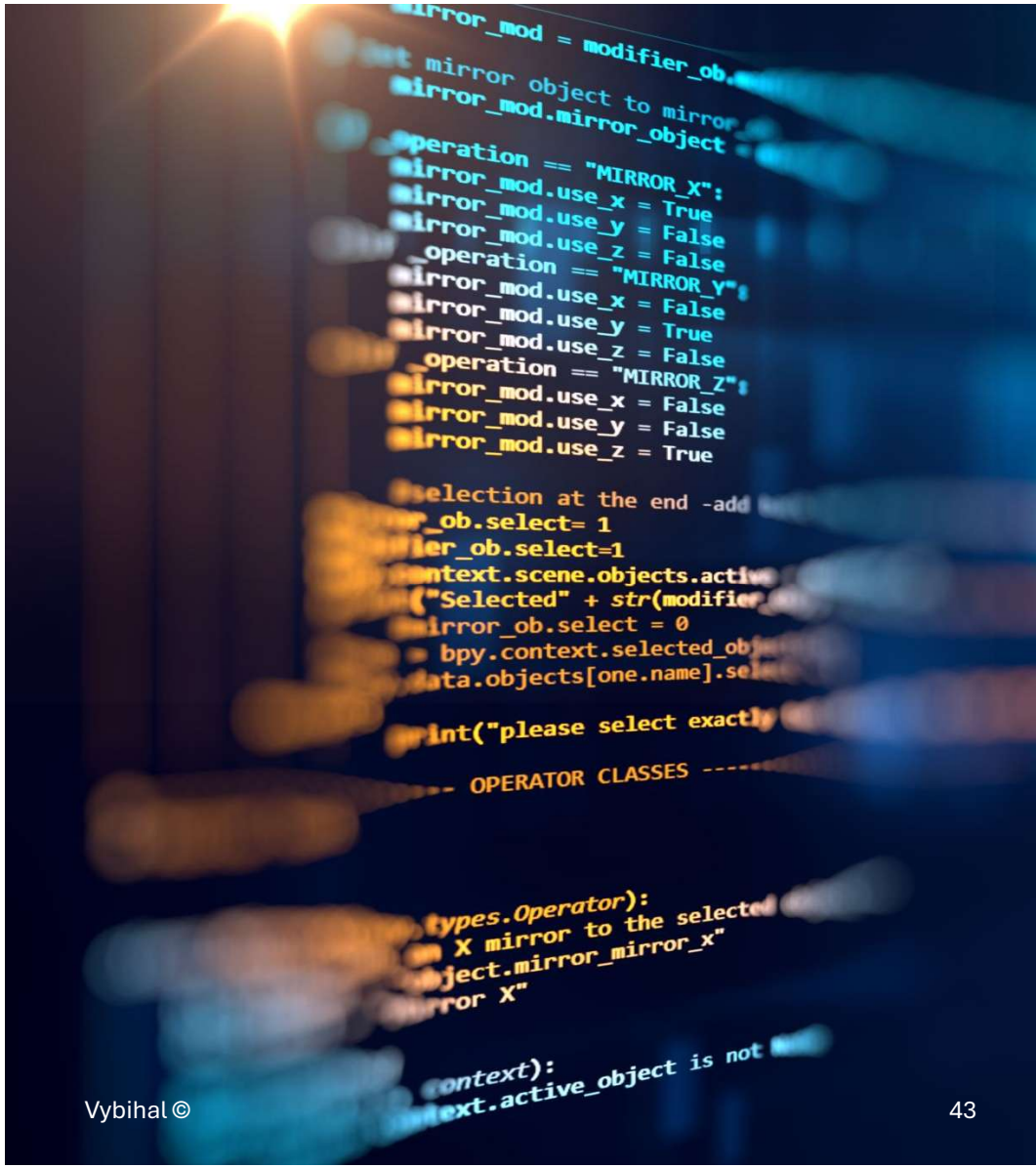
Bla bla

Return Format:

Bla Bla

Useful Tools

- UMLet (or similar): https://www.umlet.com/
- Spreadsheet software



6 Important (critical) Sections

- 6.1 Table of contents of critical sections

Title	Module	Location	Comments

- 6.2 Critical Sections
  - Title
  - Description of its importance & purpose
  - Proposed solution in words
  - Flowchart of solution (not pseudo-code)
  - Explanation of flowchart

A critical section is defined as a moment in your interview with the client when they express concern about the correctness of a particular operation the computer must accomplish.

You will probably have 0 to 10 of these in an average project.

What is critical about the following?

- Live bus app showing the position of buses?
- You download a stream of data from a device that synchronously sends two packets of information.
  Example Data1={p1, p2}, Data2={p3,p4}.

Ways to describe critical sections

- Flowcharts
- State diagrams

Best ways

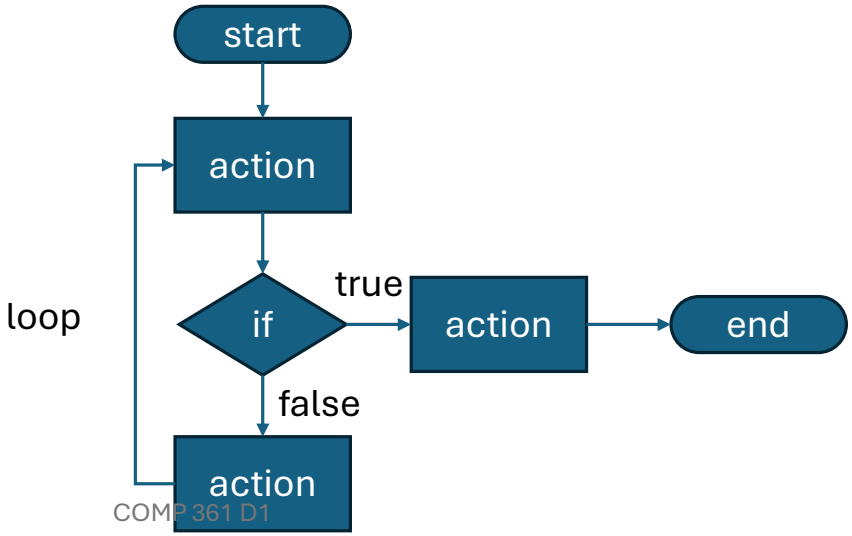
- Queries
- Interaction diagrams

Better for design document, but sometimes

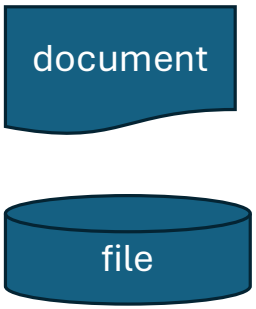
Represent the solution in a non-technical way

Flowcharts

- Provides useful information to the client in a way they understand
- Also provides guidance to a developer
- Uses traditional flowcharting techniques (not UML)
- Provide human readable text, not programming expressions



Parallel split & merge



There are other symbols, but these are the most used.

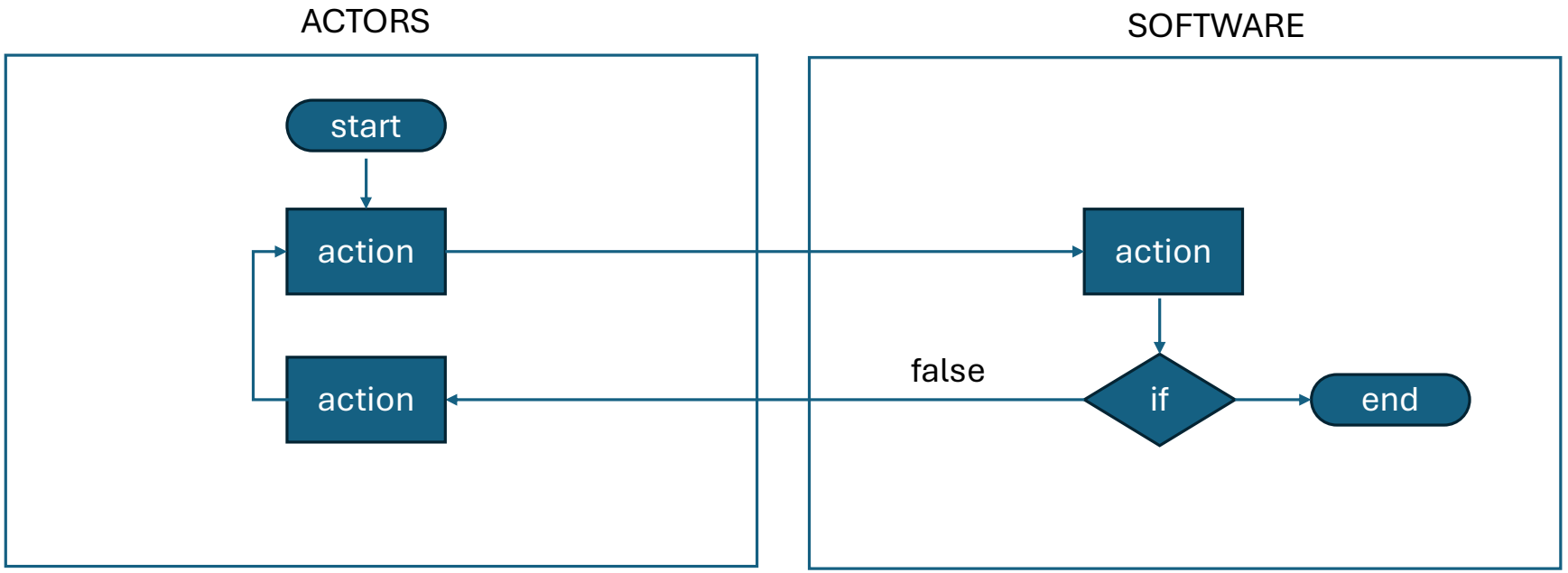
Question

- Create a flowchart for a menu that handles add, subtract, multiply, and divide. The user selects the menu option and then enters the two values to perform the calculation. The program displays the result and then returns to the menu. The menu includes a ‘Terminate Program’ option.

System Flowcharts

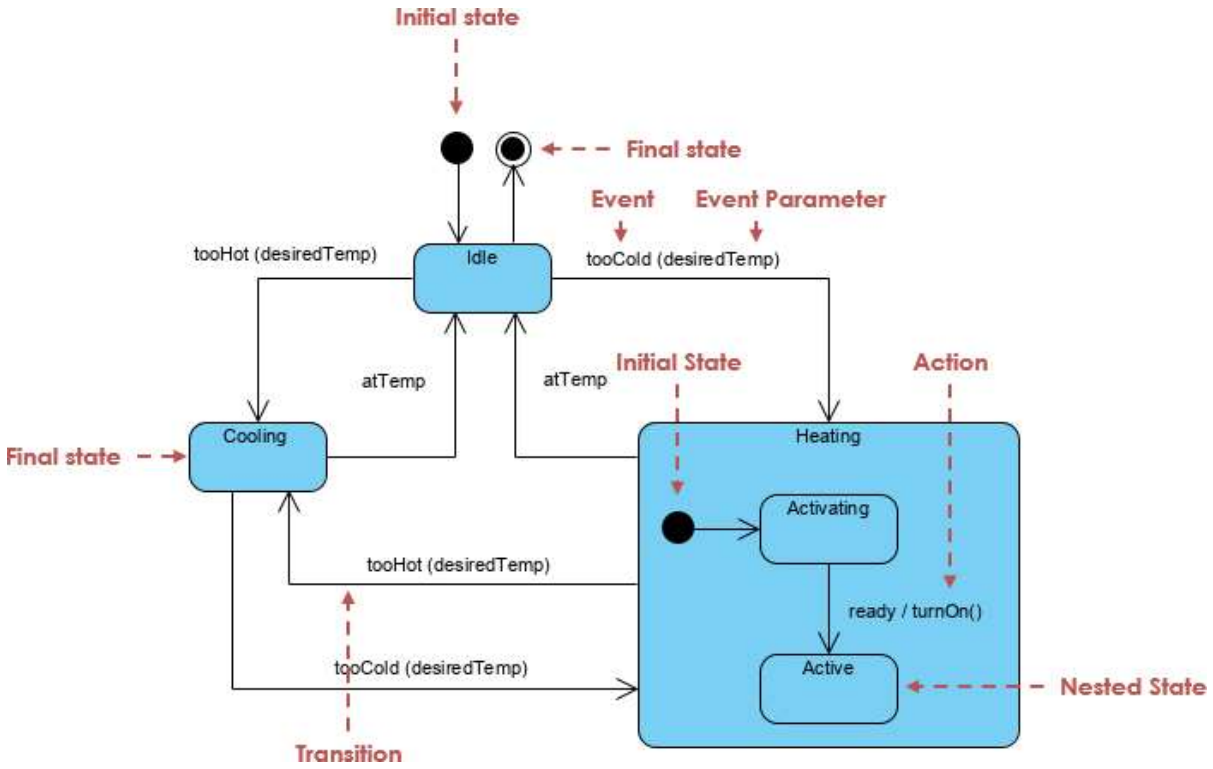
We can have multiple boxes for each actor and device.

- Defining the interaction between computer and human activities.



State Diagram

- When it is easier to talk about the events than the flow.
- Notice that there is a main system and then a subsystem. They each have an Initial State.



Question

- Use a state diagram to describe the use of a handheld calculator that only does add, subtract, multiply, divide, and equals.

Queries

- When it is easier to simply say it in words as a single structured sentence.
- Good to provide an example output given the query.
- Structured language SQL

```
/****** Script for SelectTopNRows command from SSMS  ******/
SELECT TOP (1000) [id]
      ,[code]
      ,[firstName]
      ,[lastName]
      ,[locationID]
  FROM [SampleDB].[dbo].[employees]
```

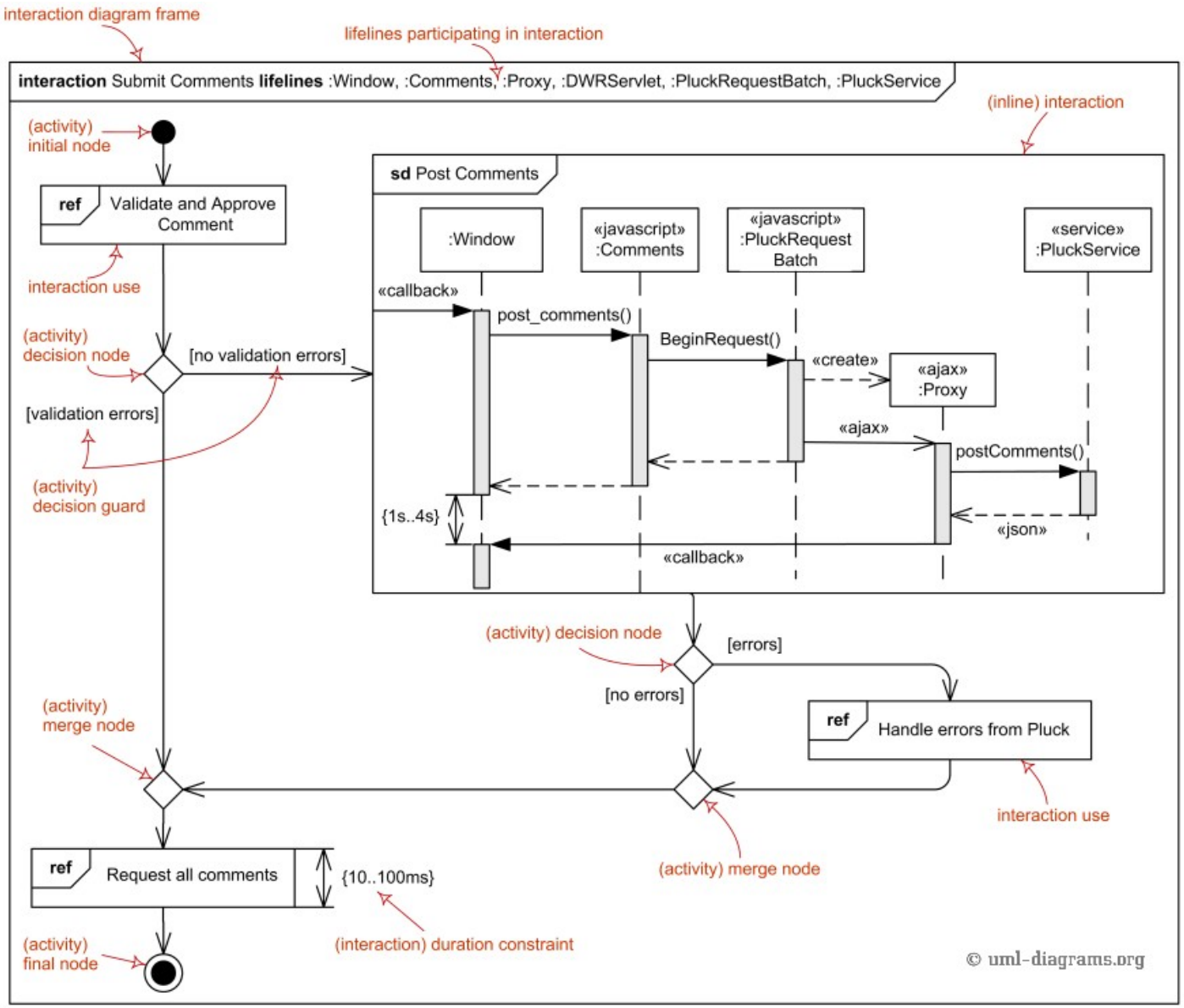
100 %

Results | Messages

	id	code	firstName	lastName	locationID
1	1	code1	firstName 1	lastName 2	1
2	2	code2	firstName 2	lastName 2	2
3	3	code3	firstName 3	lastName 3	2
4	4	code4	firstName 4	lastName 4	3

Interaction Diagram

- When a flowchart needs to speak about many interacting components.
- Use when there are multiple timing constraints and validation constraints.
- Use when you want to talk about function calls and objects.
- Ref – reference to a use-case
- Sd – could include a sequence diagram



Useful Tools

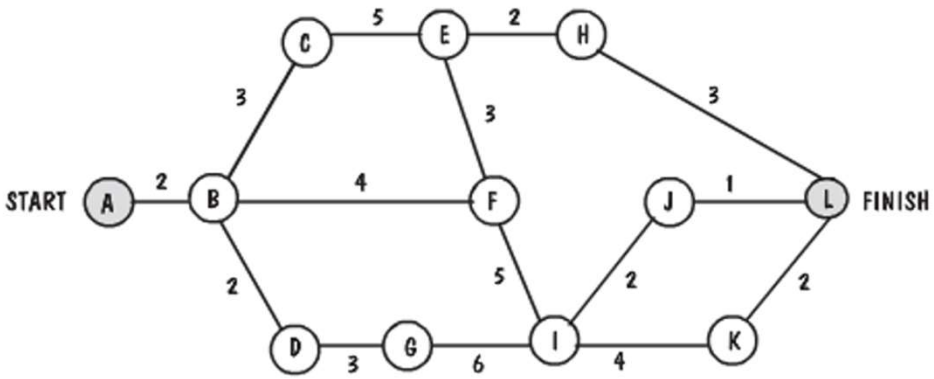
- UMLet (or similar): https://www.umlet.com/
- Lucidchart: https://www.lucidchart.com/



7 Effort Estimation

- 7.1 Optimal Workflow Graph
  - Software development workflow choice
  - What is the shortest path to complete a project?
  - What is the practical shortest path given our constraints?
- 7.2 Effort Estimation Table
  - How many hours of work will it take?
- 7.3 Gantt Chart
  - A calendar-based commitment to the actual workflow
- 7.4 Estimated Project Cost
  - Final cost breakdown that includes all project requirements

Basic Idea



If I have two programmers, in what order and how much time would it take to complete building a personal website: landing, about me, projects?

- You want to plan how much time the job will take (the Activity Graph).
- Depends on:
  - Tasks to complete
  - Number of days per task
  - Parallel work
  - Engineering workflow
    - Fast: Waterfall
    - Slow: Spiral
    - Middle to Slow: Protortype

7.1 Optimal Workflow Graph

- Two important things to identify & think about:
  - Decide on the project's workflow
  - Look at how the workflow you choose will impact the time it takes to complete the app.
    - Find the ideal shortest path to product delivery without constraints (theoretical)
    - Find the actual shortest path to product delivery given constraints (actual)

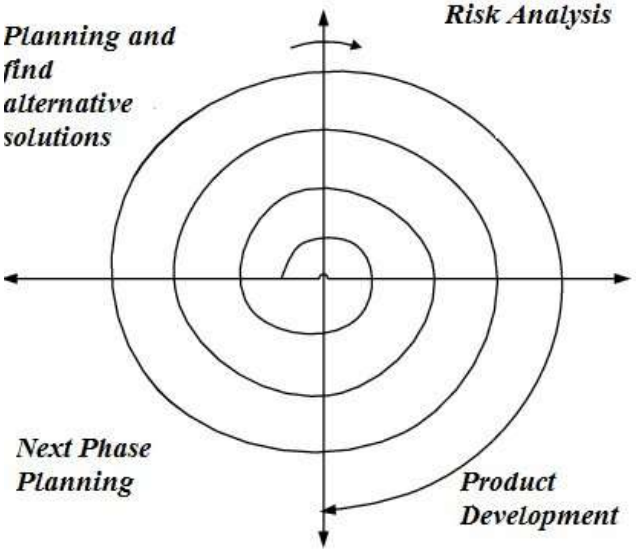
Choosing a workflow

- What is a workflow?
  - This defines the way you, the engineer, will interact with the **client** and the **developers**.
  - This is often depicted as a flowchart
  - It is an ideology
  - It is a method
  - It is a work ethic

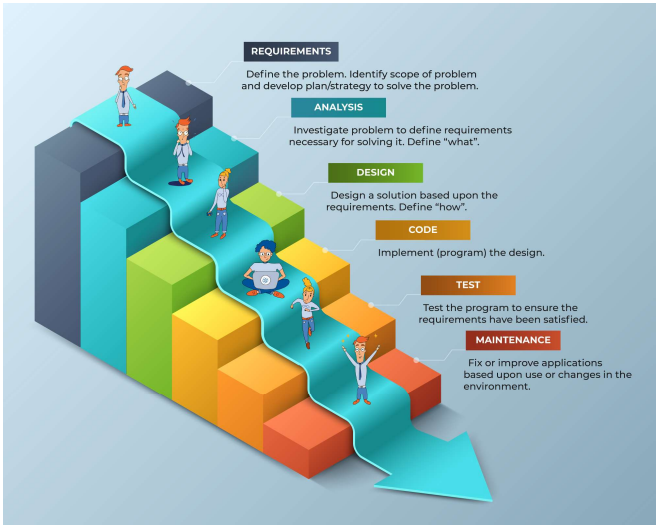
Two workflows

- A workflow describing the project's evolution
- A workflow that describes how the people will do their job
  - We will look at this next semester

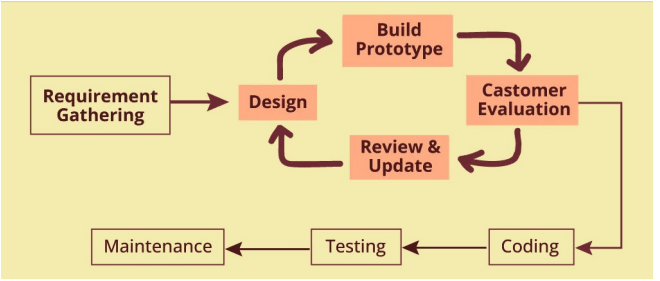
The Spiral Method



The Waterfall Method



The Prototyping Method



Project Management

Main Differences

- Waterfall Method
  - Once you finish a phase, it is done; you do not return to it.
  - Speedy development but sensitive to mistakes.
- Spiral Method
  - Phases of a project are NEVER done; you always return to them and refine them, until the project is finished.
  - Handles mistakes through gradual evolution. Great quality. Sacrifices time.
- Prototyping Method
  - Every phase of the project delivers a partially working app to the client.
  - Everyone sees your dirty laundry, but you get much feedback.

Each of these methods add or remove extra time to the project.

Choosing a Management workflow

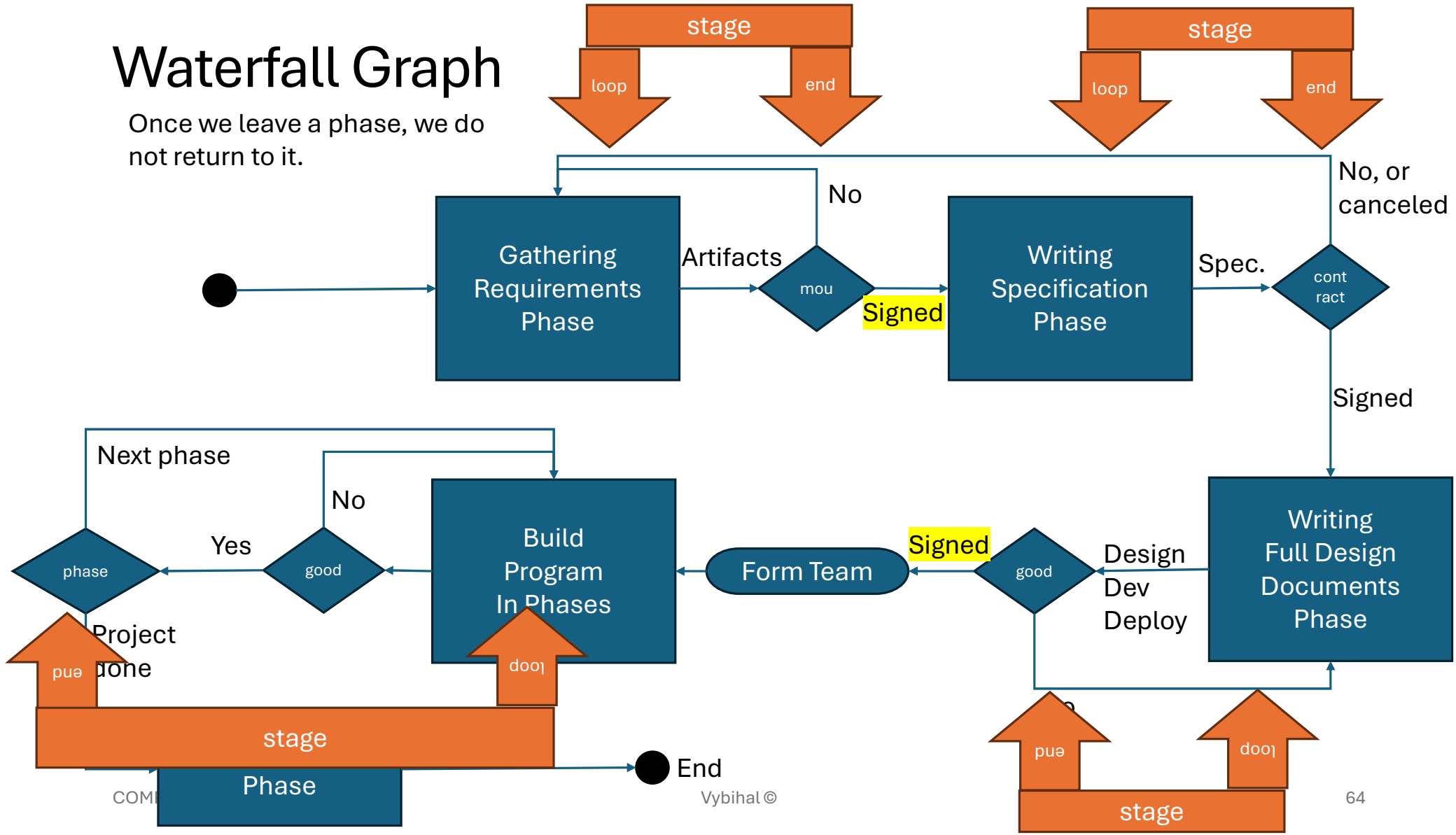
- Three major types of workflow
  - Waterfall (commonly used by business)
    - The project evolves based on documents and stages.
    - Requirements → Specification → Design → Building & Testing → Deployment.
    - Least expensive and fastest method.
  - Prototyping (Space X)
    - The project evolves by building toy versions of the final product.
    - Project Requirements → LOOP: Toy Spec & Design & Build & Test & Deploy → Feedback → Repeat toy step until done
    - You get to use it quickly.
  - Spiral (NASA)
    - Quality, correctness, and verification is top priority.
    - LOOP: Requirements, Specification, Design, Build, Test, Feedback (→ Deploy → Feedback)
    - Most expensive and slowest method but highest quality and fewest failures.

Choosing a workflow modification

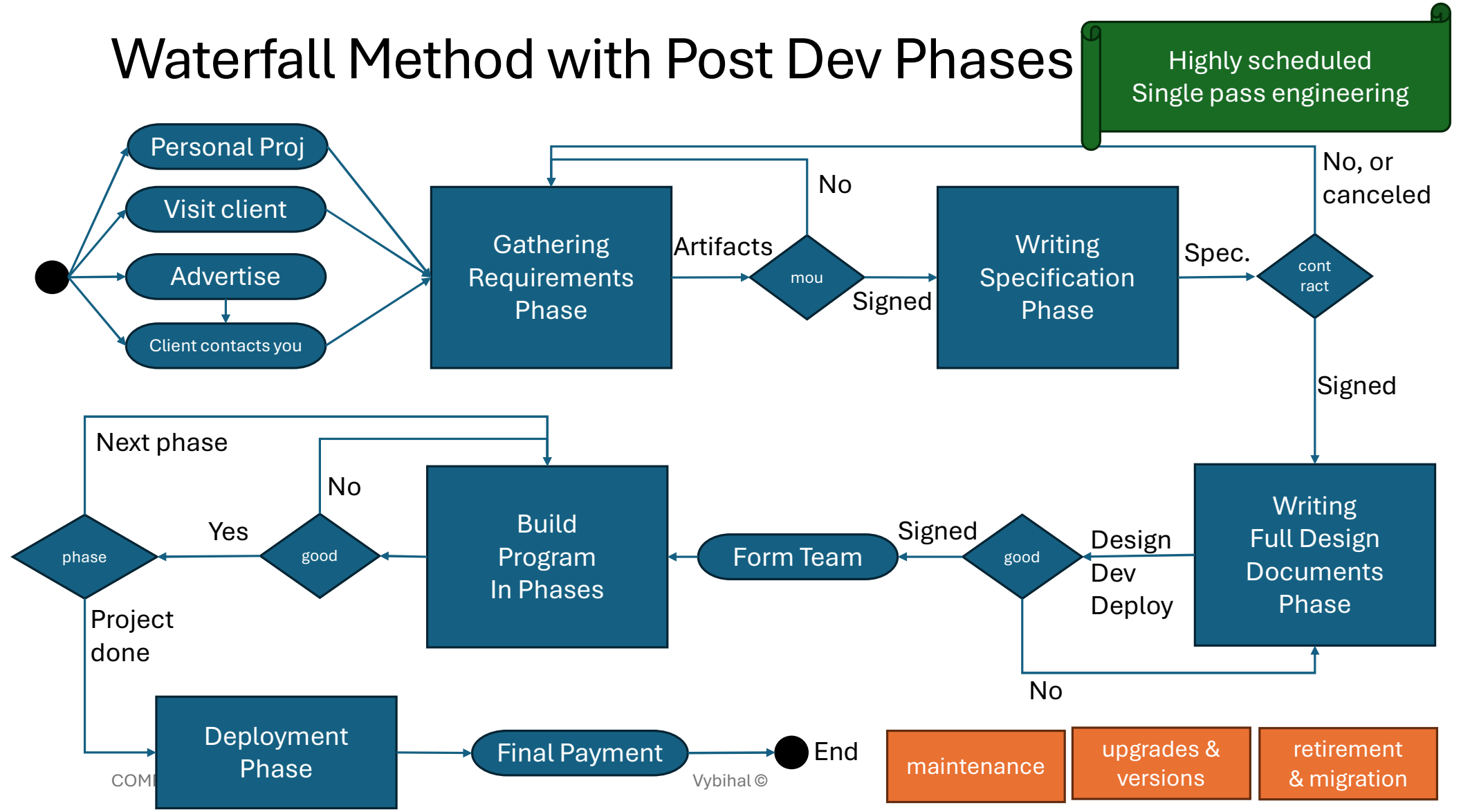
- Modifications to the three basic workflows
  - Agile
    - Motivation: We find it hard keeping to schedule, and the specifications keep changing.
    - Solution: Novel working method + Pad schedule with buffers and catchup periods.
  - Iterative by module
    - Motivation: Easier to build slowly.
    - Solution: Build and test one module at a time.
    - Result: Ensures high quality engineering at the expense to schedule (less parallel dev.).
  - Iterative by stakeholder signoff
    - Motivation: The client never likes what we build.
    - Solution: Schedule demos and signoff with stakeholders at regular periods.
    - Result: Ensures high stakeholder satisfaction at the expense to schedule. Must factor in stakeholder presentations and task rebuilding into the schedule.

Waterfall Graph

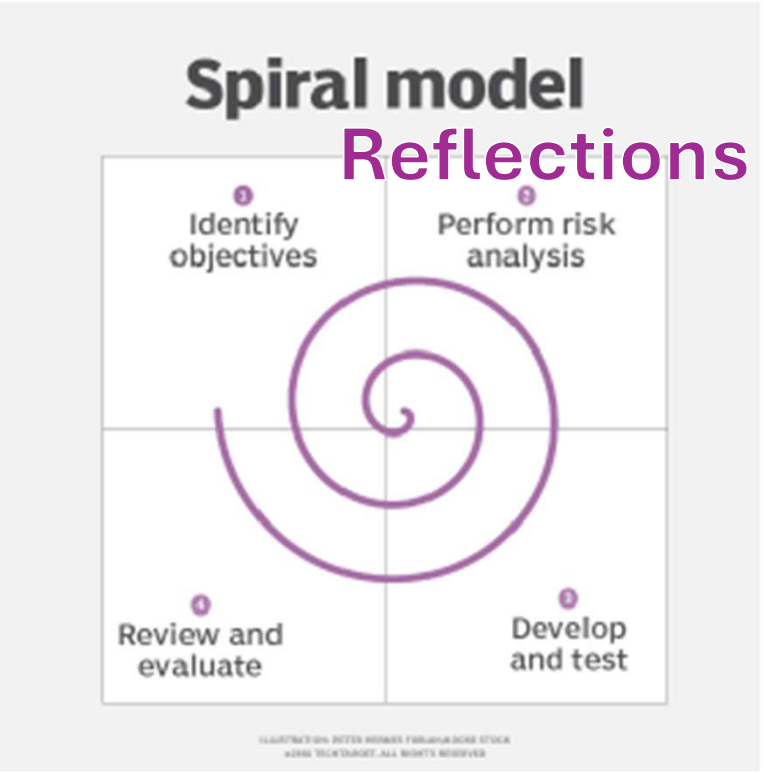
Once we leave a phase, we do not return to it.



Waterfall Method with Post Dev Phases

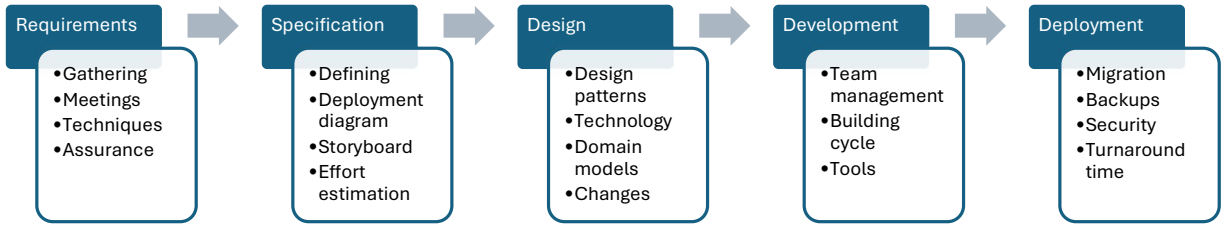


COMI Vybihal ©



Reflections

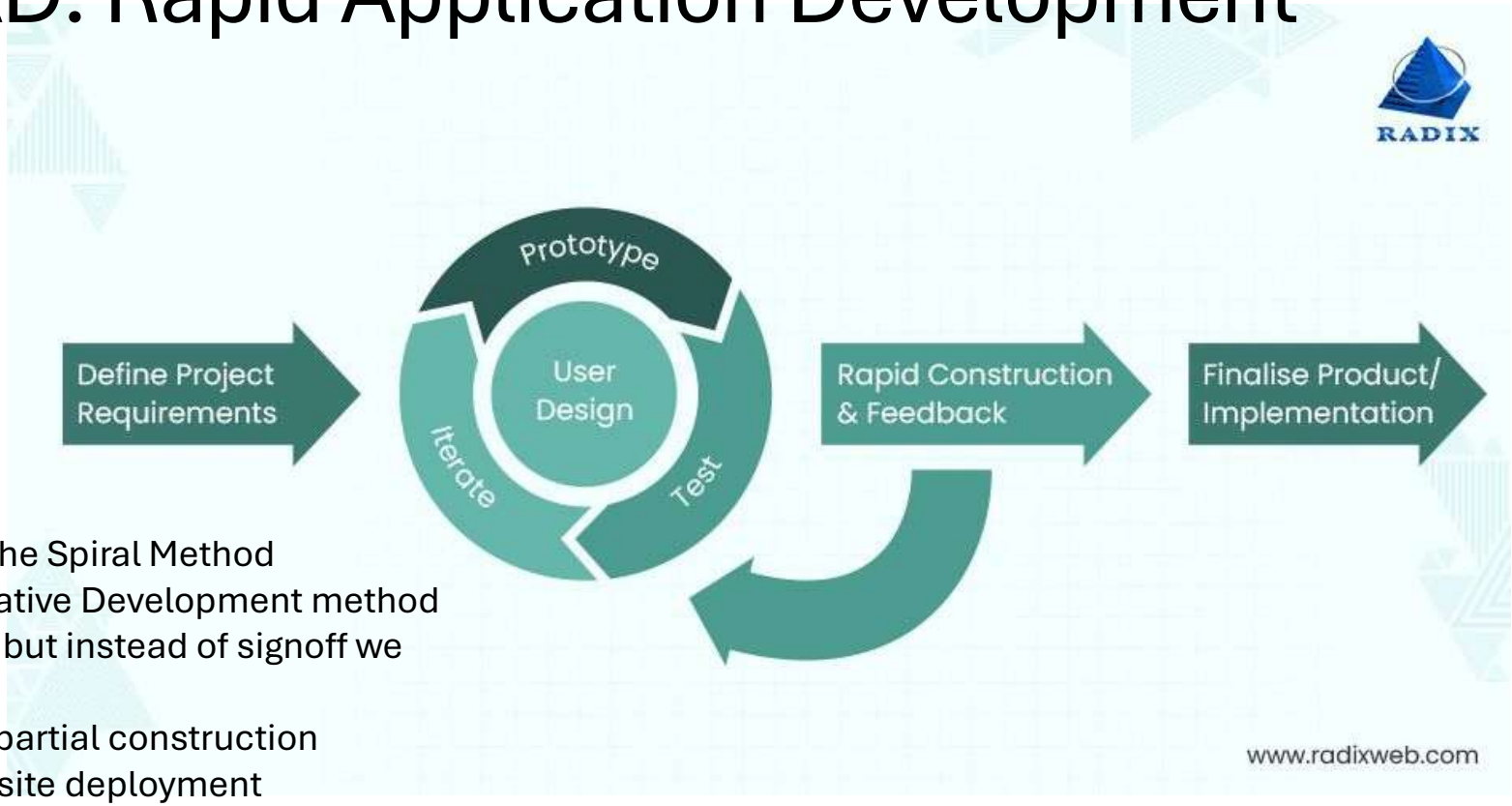
- First method to abstract the engineering process
- It views each of the stages of software development as a spiral
- We do not exit the spiral until everyone has signed off: engineers, managers, stakeholders, client, etc.
- In other words, we stay within a stage until the spiral is completed.
  - Ex: We started the Requirements phase. We do not leave requirements until we have made sure that all requirements/stakeholders/types of information we want to gather were identified, that everyone is satisfied, that we identified and resolved all missing stakeholder requirements, we had a formal review and discussion of all the gathered requirements, we developed documents of each requirement and reviewed it with the stakeholders, we made corrections and met stakeholders repeatedly, everyone signs off…. Now, we can go to the Specification phase.



Reflections at every cycle.

Precise scheduling is difficult.

RAD: Rapid Application Development

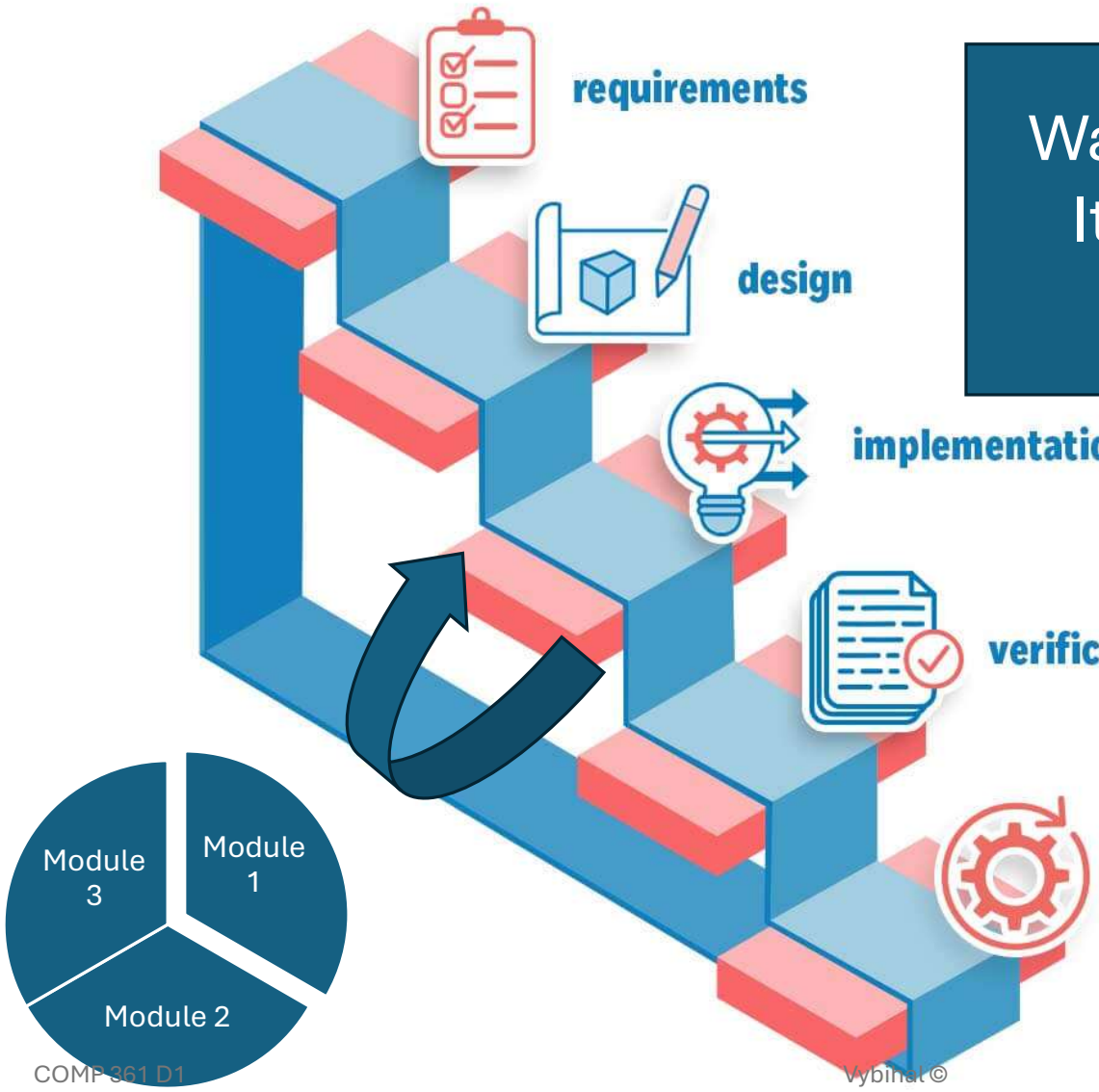


- This is not the Spiral Method
- It is the Iterative Development method with client, but instead of signoff we have:
  - Rapid partial construction
  - Client site deployment
  - Client feedback
  - Repeat & do better

Work Workflows

- Iterative by module
  - Build or plan a single module completely before advancing.
- Iterative by sign-off
  - What would the client like to build first
  - Build that
  - Get their sign-off before advancing
- Agile
  - A method to handle the dynamic changing nature of some projects

Waterfall with Iterative by Module

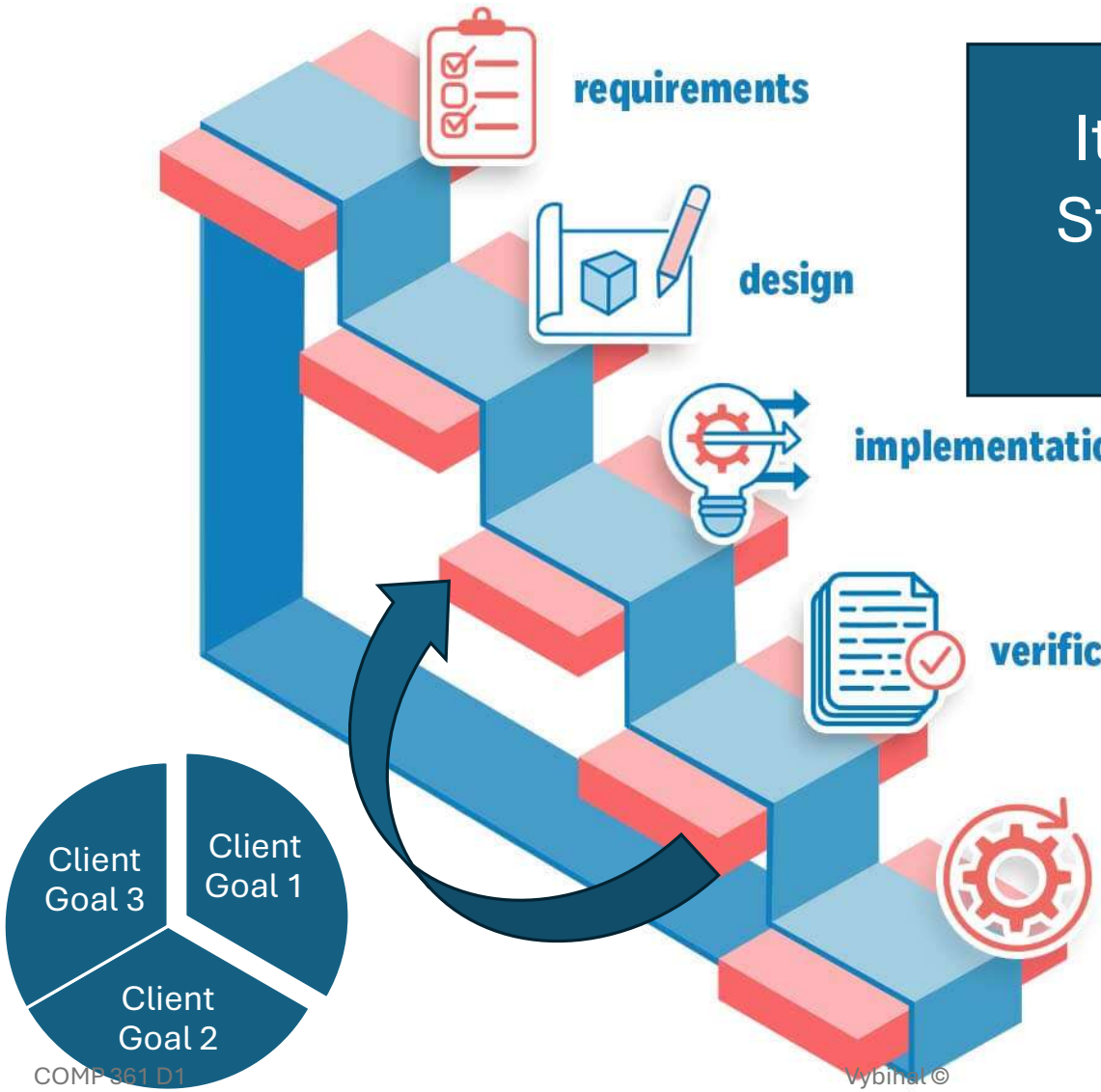


This is the waterfall method, but with a cyclic implementation phase.

Start date fixed.

End date, when all internal testing fixed.

Workflow:
Divide project into modules. Select a module to develop. Work on that module until all test cases are satisfied. Then go to next module and repeat. Make sure previously completed modules are not broken before going to next module.



Iterative by Stakeholder Signoff

This is the waterfall method, but with a cyclic implementation and verification phase.

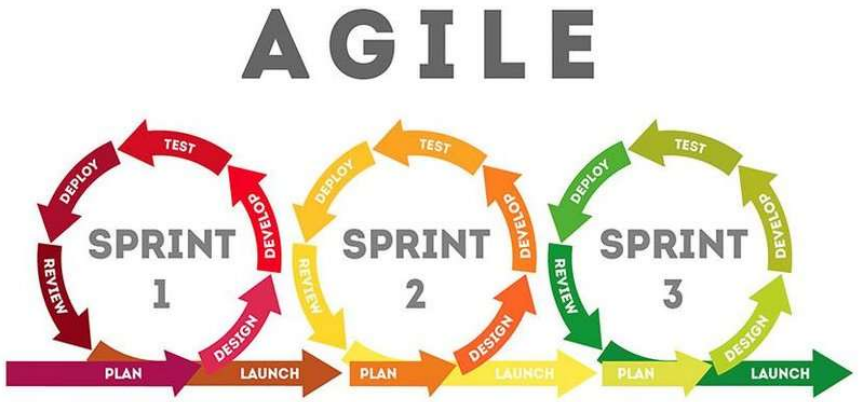
Start date fixed.

End date, when customer signs off on all test.

Workflow:
Divide project into modules. Select a module to develop. Work on that module until the client signs off satisfaction. Then go to next module and repeat. Make sure previously completed modules are not broken before going to next module.

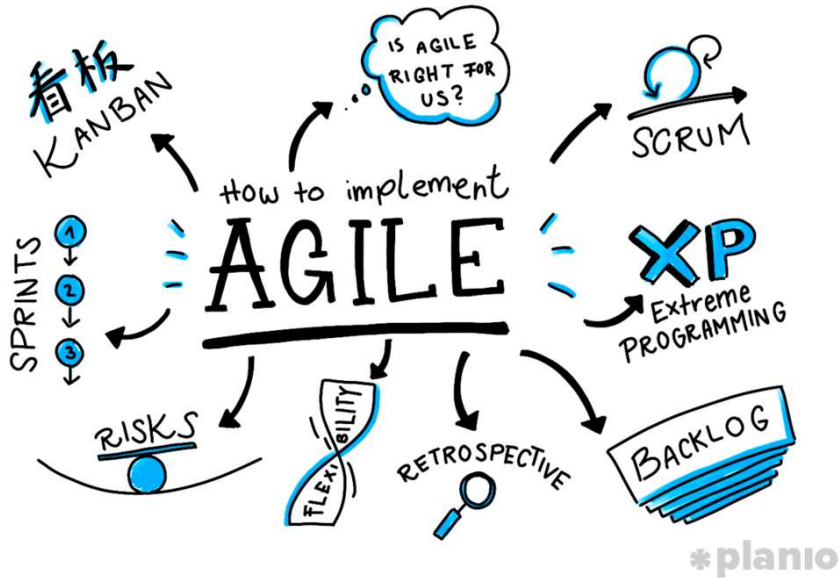
A final signoff from the stakeholder is required at the end of this cyclic phase before we can go into maintenance.

- Pick a part of the program to develop, call it Sprint n, where n >= 1 and is unique.
- For each sprint spiral all waterfall steps until done for this single sprint.
- Go to next sprint, until entire project done.
- Deliver product (unless incorporated in prototyping method)



Best during Project Management Phase

Great for projects with a lot of uncertainty but have flexibility to do things differently. **Notice**, it looks a lot like the Spiral Method. **Notice 2**, "launch" means internally.

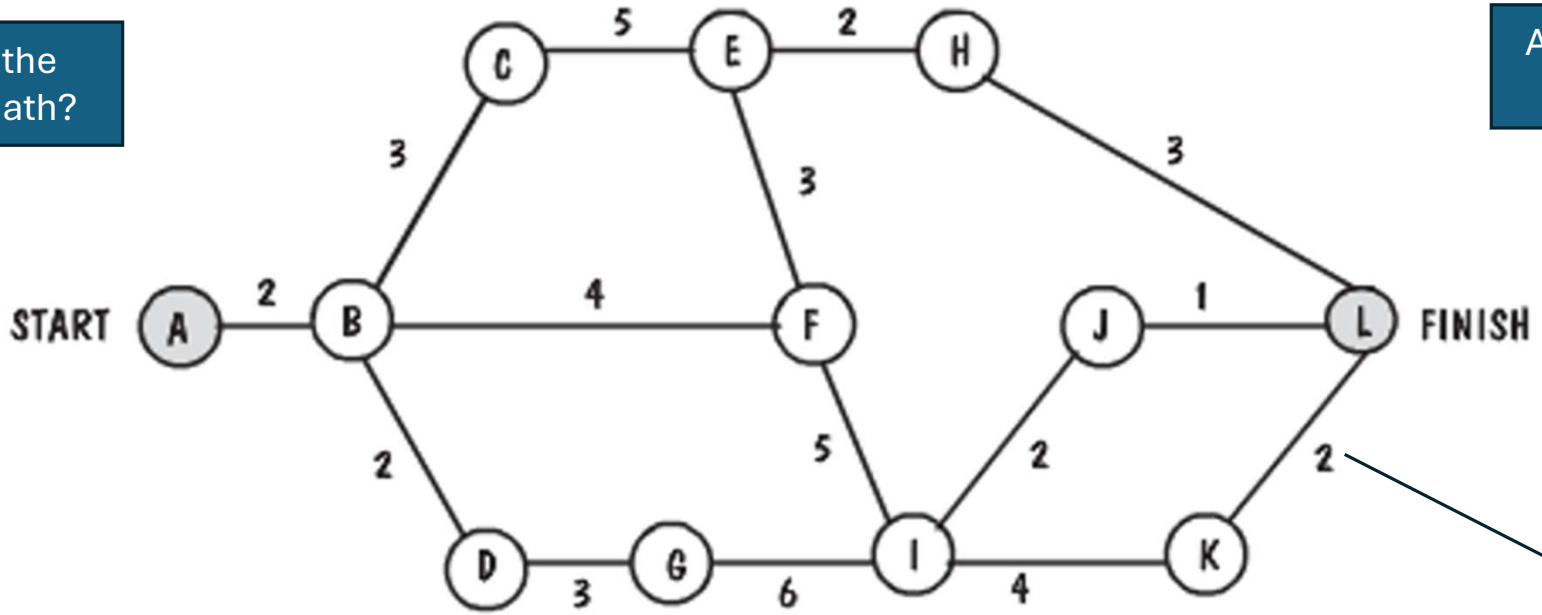


Development Techniques

We will talk about this when we explore team development.

Longest Path to Completion: Activity Graph

What is the longest path?

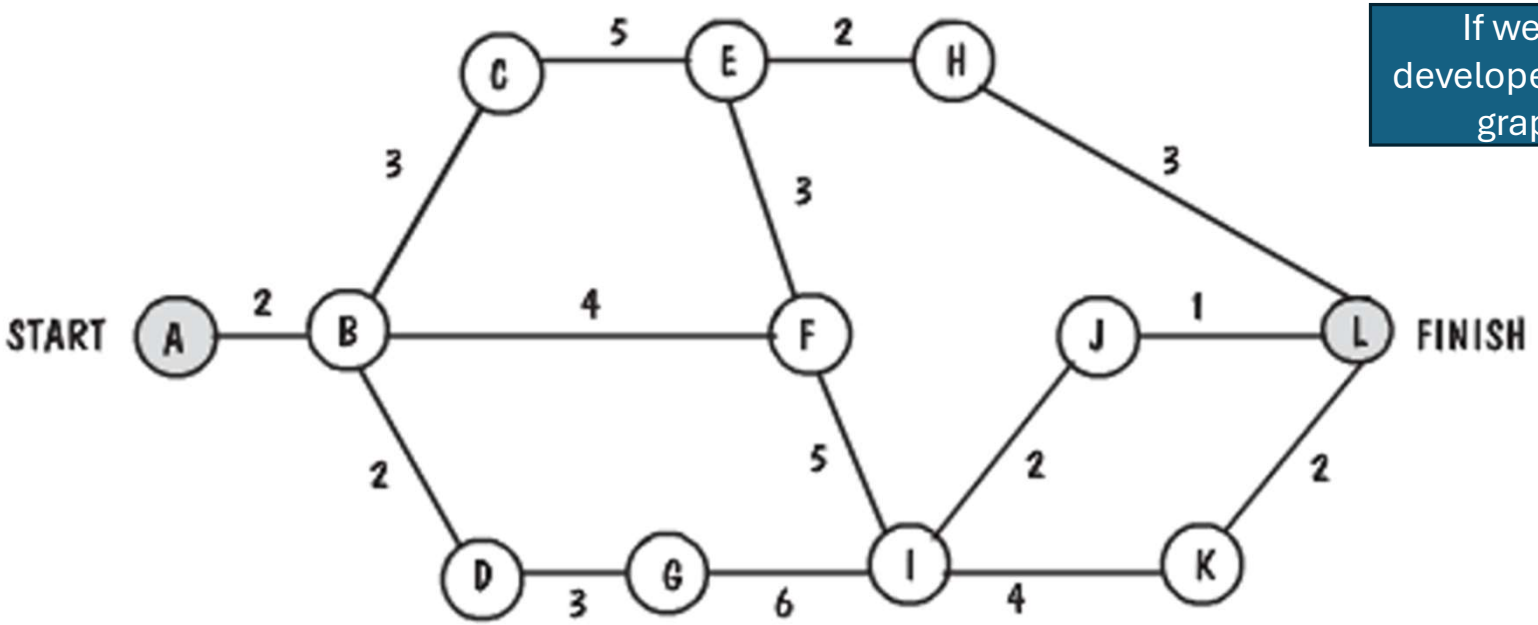


All nodes must complete

- Activity graphs can be constructed to focus only on the app alone or cover the entire development process.
- Optimization occurs by maximizing parallelism & shortening arc lengths.
- Node = either a module of the app or a phase is the development process.
- Arc = the number of hours/days it takes to complete a Node.
- This is not a cyclic graph: longest path is the earliest project can complete.

Node K takes 2 days to complete

Longest Path to Completion?



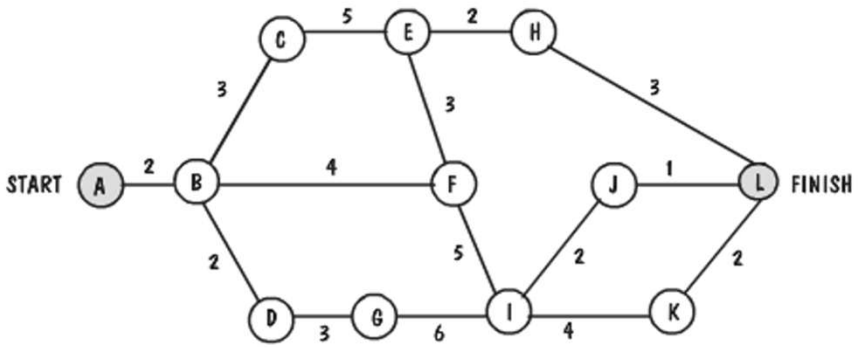
If we have only 2 developers, how will this graph change?

- The actual length of a project depends on constraints.
- Constraints: number of developers, vacation time, sickness, budget, client's compute max, etc.
- Presenting the shortest path and the practical shortest path to the client is an important discussion point.
- It is the practical shortest path that will be used in the final project Gantt Chart.

Effort Estimation

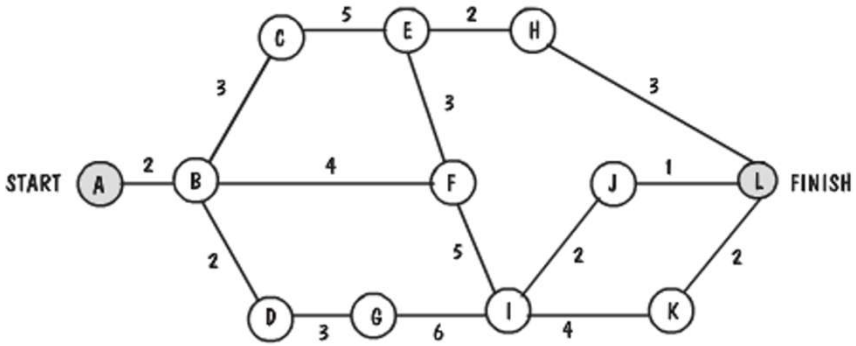
Summation of all arcs

- The activity diagram's arcs contained estimates for the duration of each step.
- Effort estimation determines the total estimated effort. For example, what is the total days of work in the graph?



- And what are all the preconditions of each node? Represented as a table.

Effort Estimation



Node	Duration	Pre-conditions
A – B	2	Met all stakeholders
B – C	3	Docker image available
B – F	4	Internet available

Total 47
Optimal (need optimal graph)
Practical (need practical graph)

- Total means the number of days or hours someone needs to work to complete the work of every node. The hours we need to pay someone.
- Optimal means the number of calendar days or hours to complete the project with an infinite number of people.
- Practical means the number of calendar days or hours with the given constraints.

Estimated project cost

Element	Unit Cost	Number of Units	Additional Costs	Subtotal
Workdays	$100/day	47 days	$0.00	$4,700.00
Team Github Pro	$500/year	1 year	$0.00	$ 500.00
Equipment	N/A			
			TOTAL:	$ 5,200.00

Note:

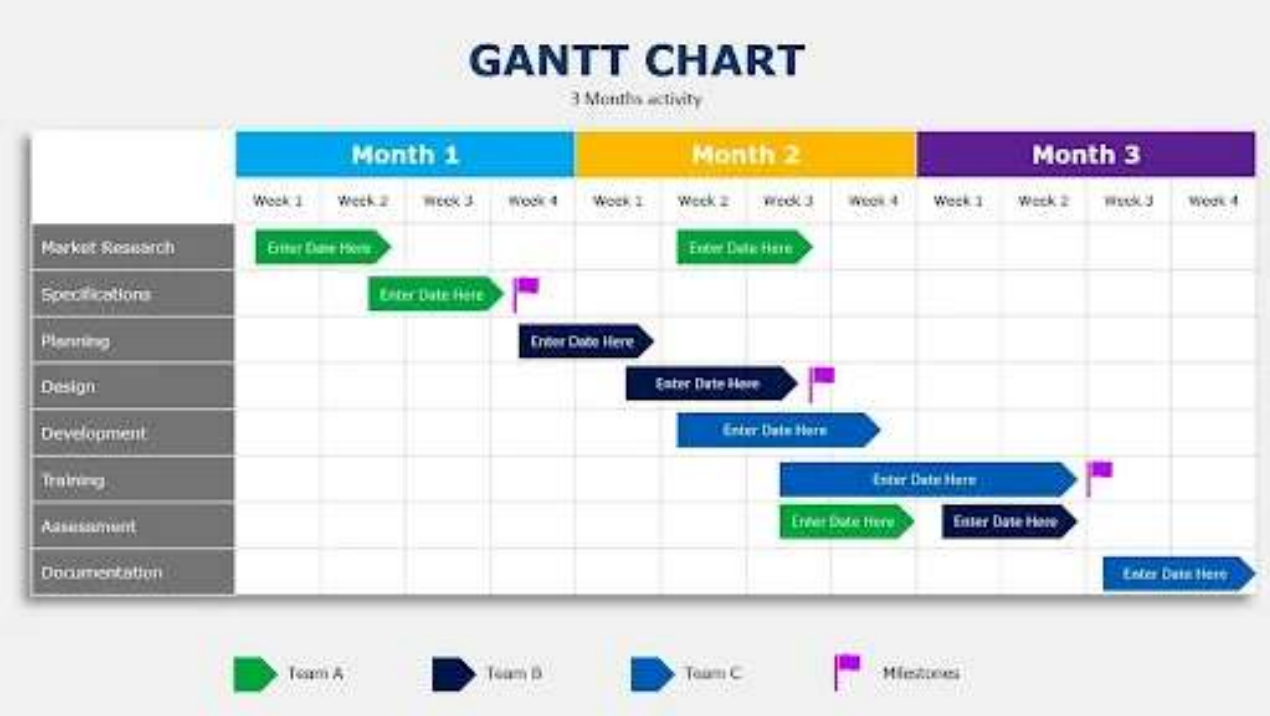
- The shortest path from the effort estimation graph is used to determine when the product can be delivered to the client.
- The estimated cost (workdays) is based on the summation of every node from the effort estimation graph.

Computing the delivery date

- To compute the delivery date, we must:
  - First, find the shortest path in the effort estimation graph taking into consideration parallel work constraints.
  - Second, we then add in padding time for unexpected problems. Usually this would be 10% more time but could be lower or greater depending on your experience.
  - Last, we then look at the calendar, for example:
    - Will you work weekends?
    - How many hours in a day will you work?
    - Will you take holidays?
    - What about sick days?
- We put all this information into a Gantt chart.

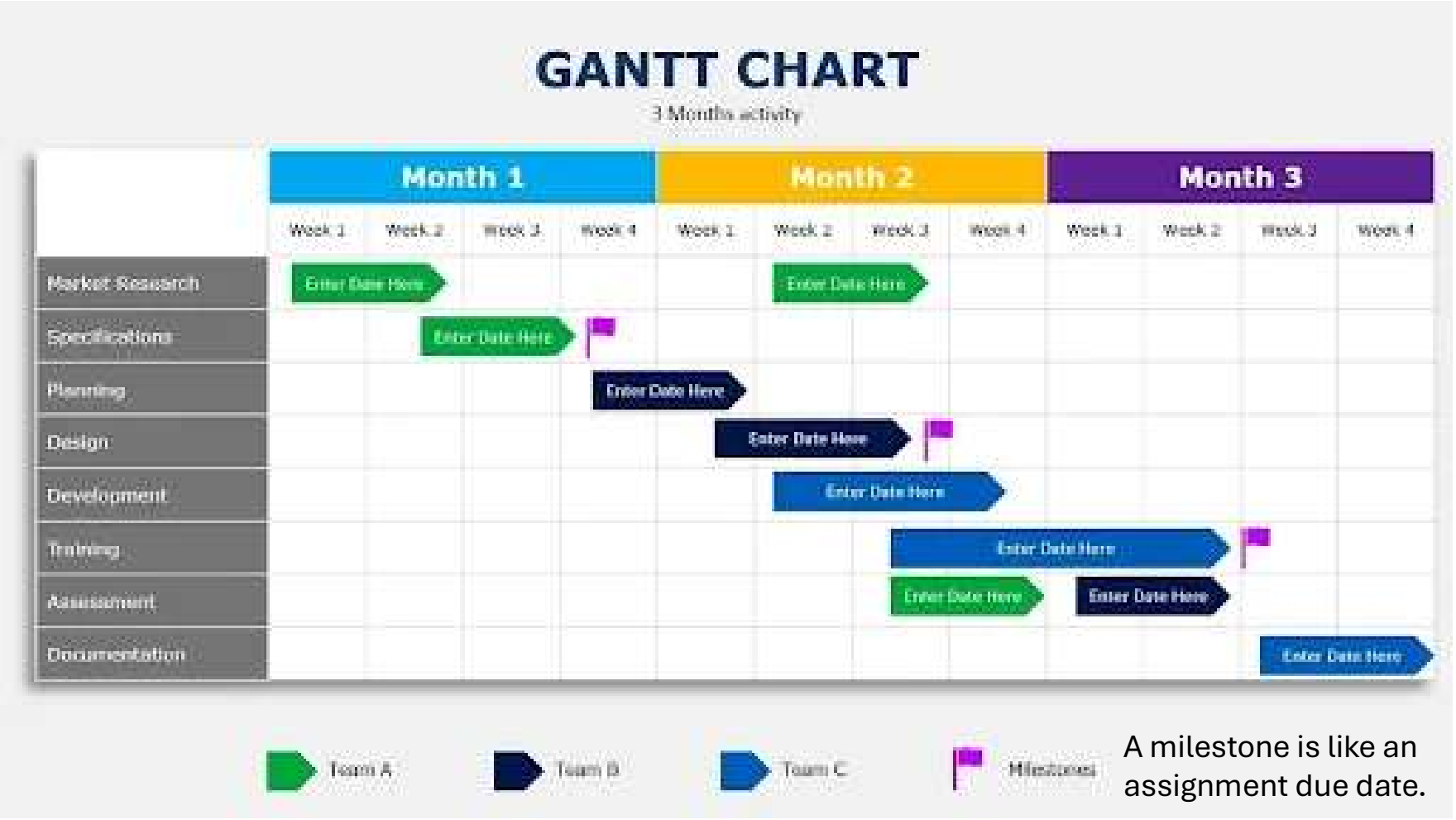
Gantt Chart

If we work only Monday to Friday, no vacations or holidays, how many calendar workdays is 47 days?



Great for the team leader and the client to track the state of the project.

- Based on actual calendar
  - Our 47 days includes weekend, holidays, and vacations.
- Rows
  - Project phases, and
  - Software modules
- Columns
  - Actual start & end dates
- Flags
  - Important milestones
  - $ penalties may apply
- Allows for parallelism
- Allows for teams
- Know where you are/should be



A milestone is like an assignment due date.

People in different colors.

Demo Flag,
Deliverable Flag

Useful Tools

- Spreadsheet
- UML diagram programs
- Top 10 Gant Chart Makers